



Graduation Project, Part-I (SWE 496)
Software Engineering Department
CCIS, KSU

Project Advisor:
Dr.Sultan Alsarra

Submitted by

NO.	Name	ID
1	Moath Alonayq	444100678
2	Tariq Algadheeb	444100905
3	Mohammed Albilaly	444101589
4	Saad Alqarni	444102880

Date submitted
00/00/2026

ABSTRACT

The rapid adoption of AI tools like ChatGPT and Gemini in workplaces has created a serious risk of unintentional data leakage, as employees unknowingly share sensitive information such as passwords, API keys, and personal data with these platforms.

Hares AI is a browser extension that intercepts user input before submission, analyzes it locally on the device, and alerts or blocks the action if sensitive data is detected without sending raw sensitive data to external servers.

The goal of this project is to provide a lightweight, privacy-preserving solution that protects organizations from AI-related data exposure.

Table of Contents

1. Introduction	2
2. Domain Analysis	2
3. Risk/Constraints	3
4. Project Plan	4
5. Quality Assurance Plan	5
6. Requirements	6
7. Problem Complexity	14
8. System Use-Cases	14
9. Analysis Class	26
10. Interaction Diagram	28
11. Design Class	32
12. System Architecture	39
13. User Interface Mockup	42
14. Database Schema	60
15. Algorithms	62
16. Expected Deployment	71
17. Test Scenario	74
18. Project Status	78
19. Conclusion	79
20. Reference	80

Table of Tables

Table 1. Comparison of Features and Limitations in Related Systems.....	3
Table 2. Risk Assessment Matrix.....	4
Table 3. Test Cases.....	78
Table 4. Project Status.....	79
Table 5. Glossary of Terms.....	82

Table of Figures

Figure 1. Project Gantt Chart.....	5
Figure 2. User Account Management.....	15
Figure 3. Prompt Interception & Detection use case.....	16
Figure 4. Logging, Audit & Prompt Assistance.....	17
Figure 5. Dashboard Management use case.....	18
Figure 6. Hares AI Analysis Class.....	27
Figure 7. Login Interaction Diagram.....	29
Figure 8. Registration Interaction Diagram.....	29
Figure 9. Review Logs Interaction Diagram.....	30
Figure 10. Organization Dashboard Interaction Diagram.....	31
Figure 11. Revision Prompt Interact.....	32
Figure 12. Hares AI Class Diagram.....	38
Figure 13. System Architecture.....	39
Figure 14. Landing Page.....	44
Figure 15. Register Page.....	44
Figure 16. Login Page.....	45
Figure 17. Forget Password Page.....	46
Figure 18. User Dashboard Page.....	47
Figure 19. User Activity Log Page.....	48
Figure 20. User Blocked Prompt Page.....	49
Figure 21. User Revision Prompt Page.....	49
Figure 22. User Profile Page.....	50
Figure 23. Department Admin Dashboard Page.....	51
Figure 24. Department Admin Log Page.....	52
Figure 25. Department User Management Page.....	53
Figure 26. System Admin Dashboard Page.....	54
Figure 27. System Admin Full Log Page.....	55
Figure 28. System Admin User Management Page.....	56
Figure 29. System Admin Role Assignment Page.....	57
Figure 30. System Departments Management Page.....	58
Figure 31. High Risk POP-UP Message.....	59
Figure 32. Medium Risk POP-UP Message	59
Figure 33. Low Risk POP-UP Message.....	60
Figure 34. Revision Prompt UI.....	60
Figure 35. ER Diagram.....	61
Figure 36. Database Schema Diagram.....	62
Figure 37. System Deployment Diagram.....	72

1. Introduction

The rise of artificial intelligence tools has fundamentally changed how people work. Platforms such as ChatGPT, Google Gemini, and Claude are now deeply embedded in daily workflows across industries helping employees write reports, generate code, analyze data, and make decisions faster than ever before.

However, this convenience comes with a hidden cost. Employees routinely paste sensitive information into these platforms without considering the consequences. API keys, passwords, personally identifiable information (PII), confidential contracts, and proprietary source code are all being shared with external AI servers on a daily basis. In most cases, this happens unintentionally the user simply does not realize the risk.

Organizations are increasingly aware of this threat, yet few practical solutions exist that operate at the point of input where the risk actually begins. Existing security tools focus on network monitoring or post-incident response, leaving a critical gap in real-time prevention.

Hares AI addresses this gap directly. It is a browser extension that sits silently between the user and any AI platform, intercepting input before it is submitted, analyzing it locally on the user's device, classifying the risk level, and immediately alerting or blocking the action when sensitive data is detected. No raw sensitive data leaves the device during this process. In cases where Regex-based detection is inconclusive, the system sends only the ambiguous text segment after masking any previously identified sensitive content with placeholder tags to the Gemini API for further classification, ensuring that no exposed sensitive data is transmitted externally.

The objective of this project is to build a lightweight, accurate, and privacy-preserving tool that protects organizations from AI-related data leakage without disrupting the user's workflow. The system is designed to be simple enough for any employee to use, yet powerful enough to meet the security needs of modern organizations.

2. Domain Analysis

The increasing reliance on AI tools in professional environments has created a pressing need for data protection solutions. While the market has begun to respond to this challenge, most existing approaches remain incomplete or inaccessible to the average user. After conducting a thorough review of the current landscape, we have identified four key competitors that share a similar problem space with our system, Hares AI.

System	Key Features	Limitations
LLM Guard	Local processing, Free & open-source, Blocks submission, Detects emails/API keys/passwords/SSNs	ChatGPT only, Regex-based detection, No risk classification, No admin dashboard
Nightfall AI	ML-based detection, Supports ChatGPT/Gemini/Copilot, Admin	Paid enterprise only, Cloud-based analysis, Complex to deploy

	dashboard, Blocks submission	
LeakEye PII Shield	Local processing, Supports 5+ AI platforms, Multiple protection modes, Detects wide range of PII	No admin dashboard (free), No risk classification, Early development
Lakera	Fully local processing, Free & open-source, Customizable detectors	ChatGPT only, Regex-based detection, No admin dashboard, Alert only (no blocking)
Hares AI	Local processing, Supports Multiple AI platforms (ChatGPT, Gemini, Claude), Risk classification (Low/Medium/High), Admin dashboard, Free	Requires internet for AI-based analysis, Limited to browser-based AI platforms, Detection accuracy depends on model quality

Table 1. Comparison of Features and Limitations in Related Systems.

3. Risk/Constraints

3.1 Constraints

3.1.1 Local Detection Performance: A primary constraint is the limited computational power of client-side browsers. Since Hares AI performs its primary detection locally using Regex patterns, the detection engine must be lightweight and optimized to avoid impacting the user's browsing performance. For ambiguous cases, the system relies on the Gemini API rather than local AI models, which introduces a dependency on network availability but avoids the memory and processing overhead of running language models in the browser.

3.1.2 Privacy-First Policy Compliance: The system is constrained by a strict privacy-first requirement. No raw text or sensitive data can be transmitted to the Admin Dashboard; only anonymized metadata and risk classifications may be sent, which limits the depth of centralized analytics. In ambiguous detection cases where Regex alone cannot determine sensitivity, only pre-masked text segments are sent to the Gemini API for classification. All structured sensitive data identified by Regex such as API keys, passwords, and national IDs is replaced with placeholder tags (e.g., [API_KEY], [PASSWORD]) before any external transmission.

3.1.3 Browser Security Sandboxing: Operating as a browser extension limits our access to the system's underlying hardware. We are constrained by the Manifest V3 security standard, which strictly controls how we can intercept user input across different AI platforms like ChatGPT, Gemini and Claude.

3.1.4 Real-time Latency (Usability): To ensure high usability for diverse users, the local Regex-based analysis must occur in near real-time. We have a hard constraint of a 200ms processing window for local detection; exceeding this would degrade the user experience and encourage users to disable the extension. For ambiguous cases that require secondary analysis via the Gemini API, the total processing time including the API round-trip shall not exceed 1.3 seconds.

3.2 Risks

#	Risk	Type	Severity	Likelihood	Management Study
1	Detection Inaccuracy (False Positives/Negatives)	Technical	High	Medium	Combine Regex with AI-based detection and continuously test against diverse datasets to reduce false positives and negatives
2	Technical Skill Gap (Gemini API & Manifest V3)	Technical	High	Medium	Team training through online courses and hands-on practice with Gemini API integration and Chrome extension development under Manifest V3
3	Dynamic DOM Maintenance	Technical	Medium	High	Monitor AI platform updates regularly and implement flexible DOM selectors that adapt to interface changes
4	Market Competition and Adoption	Market	Medium	Medium	Focus on unique value proposition: free, local processing, and full AI platform coverage that competitors lack

Table 2. Risk Assessment Matrix.

4. Project Plan

The development of Hares AI is divided into sequential phases spanning approximately three months, from February 2026 to May 2026. Each phase builds on the previous one, starting from research and planning, moving through analysis and design, and concluding with testing and documentation.

The full project timeline and task breakdown are illustrated in Figure 1.

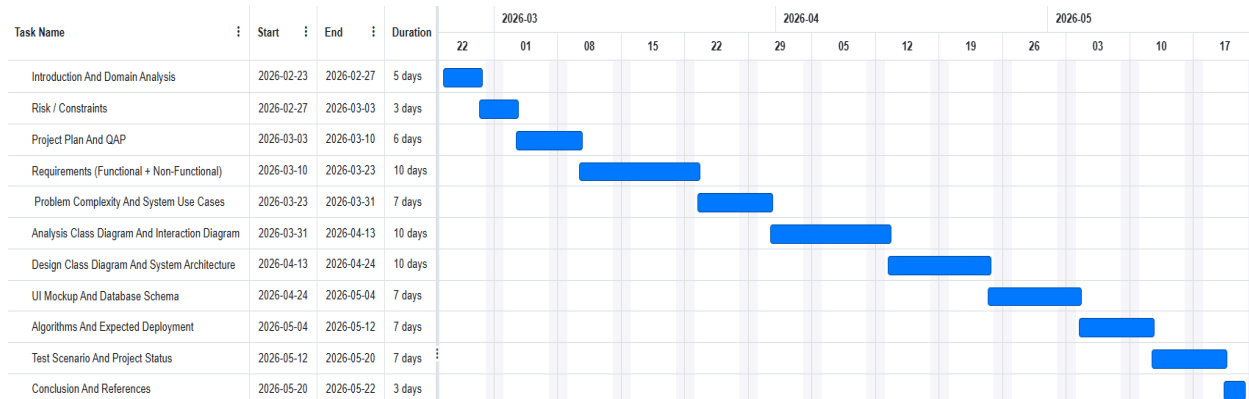


Figure 1. Project Gantt Chart.

5. Quality Assurance Plan

This plan's primary goal is to maintain high security, privacy, and performance standards for Hares AI. This is achieved by systematically managing inspections, reviews, and testing across the entire Software Development Life Cycle (SDLC).

5.1 Inspections: Internal, rigorous checks, called Inspections, are conducted by the team members (excluding the advisor) to proactively identify defects before they advance to the formal review stages.

5.1.1 Process: At the end of each major phase (Requirements, Design, Prototyping), each member's work is reviewed by the others.

5.1.2 Project Specific/Hares AI Focus: Inspections will specifically target the Regex patterns and the Gemini API integration. This is to ensure precise identification of sensitive data, proper masking of detected content before external transmission, and optimal browser performance.

5.1.3 Documentation: Findings are documented using a standardized template, and a formal written report detailing corrective actions is mandatory for subsequent follow-up.

A sample written inspection report can be put in appendix in the project report as a proof that students conducted inspection sessions.

5.2 Formal Reviews: After internal inspections, formal reviews are conducted, involving the project team, the project advisor, and potentially the examination committee.

5.2.1 Requirements Review: The system's definition of all categories (such as PII and PCI) must be verified to ensure proper alignment with the organization's safety rules.

5.2.2 Design & Architecture Review: A critical evaluation of the system's privacy-first data

handling to ensure no raw sensitive data leaks to external servers, and that all text segments are properly masked before any external API transmission.

5.2.3 Approval: A formal report documenting all findings is filled out by the advisor as proof of quality completion.

The written report can be put in appendix in the project report as a proof that students conducted formal review sessions with their adviser and eventually the exam committee members.

5.3 Testing: We follow standard procedures to ensure the software matches the design and meets user expectations.

5.3.1 Verification: Using checklists and demonstrations to ensure that the code built strictly follows the privacy-first architecture defined in the design phase.

5.3.2 Validation: Ensuring the software is secure, reusable, and performs as expected in real-world scenarios.

6. Requirements

Glossary:

AI Platform: A web-based tool that uses artificial intelligence to generate text responses, such as ChatGPT, Gemini, and Claude.

Sensitive Data: Any information that, if exposed, leaked, or misused, may cause harm to an individual or organization (e.g., identity theft, financial fraud, account compromise, privacy violations, legal risks, or security breaches).

Sensitive information categories:

1. **Contact Information:** Includes emails, phone numbers, fax numbers, social media handles, and usernames.
2. **Personal Identity Data:** Includes national IDs, passport numbers, residency/iqama numbers, driver license numbers, employee/student IDs, and insurance IDs.
3. **Financial & Banking Data:** Includes credit/debit card numbers, CVV/CVC, expiry dates, IBAN, bank account numbers, SWIFT/BIC codes, transaction IDs, payment references, and invoice numbers.

4. Authentication & Security Secrets: Includes passwords, PIN codes, OTP codes, session IDs, cookies, authorization headers, bearer tokens, API keys, JWT tokens, private key headers, and SSH keys.
5. Network & System Data: Includes IP addresses, IPv6 addresses, MAC addresses, hostnames, domain names, ports, network paths, VPN configuration fragments, and CIDR/subnet details.
6. Web Links & Identifiers: Includes URLs, URL parameters, tracking IDs, redirect links, deep links, and webhook URLs.
7. Location Data: Includes GPS coordinates, plus codes, map links, postal codes, PO box numbers, and street addresses.
8. Health / Medical Data: Includes medical record numbers (MRN), prescription IDs, insurance policy numbers, ICD-like codes, and medical keywords such as diagnosis/medication.
9. Legal & Government Data: Includes case numbers, court reference IDs, license numbers, permit numbers, vehicle registration numbers, and customs declaration numbers.
10. Vehicle Data: Includes license plate patterns, VIN numbers, engine numbers, and chassis numbers.
11. Corporate & Document Identifiers: Includes tax numbers, VAT IDs, commercial registration numbers, contract numbers, purchase order numbers, ticket numbers, serial numbers, and asset tag IDs.
12. File & Path Data: Includes Windows/Linux file paths, file extensions, file hashes, and filenames that may reveal configuration or secret files (such as .env).
13. Digital Fingerprints & Technical Identifiers: Includes UUID/GUID, hash values, device IDs, IMEI, IMSI, SIM IDs, and advertising IDs.
14. Shipping & Logistics Data: Includes shipment tracking numbers, order IDs, delivery reference codes, and shipment identifiers.
15. Cryptography & Certificates: Includes PEM certificates, public/private keys, signature blocks, certificate fingerprints, and CSR blocks.
16. Account & Application Data: Includes customer IDs, membership IDs, subscription IDs, license keys, activation codes, and account identifiers.
17. Dates & Time Data: Includes birth dates, expiry dates, timestamps, ISO datetime formats, and Unix timestamps.
18. Keyword-Based Sensitive Patterns: includes keywords such as password, secret, token, api_key, Authorization, SSN, ID and DOB.
19. Logs & Headers Data: Includes email headers, HTTP headers (cookies/authorization), user-agent strings, log IDs, and stack traces that may contain secrets.
20. Numeric Sensitive Data: Includes monetary amounts, long numeric sequences, reference numbers, receipt numbers, and barcode values.
21. Developer & Code Secrets: Includes .env variables, database connection strings, database URIs, Firebase keys, GitHub tokens, npm tokens, and similar secrets.
22. Education-Related Identifiers: Includes student numbers, university IDs, exam IDs, and course codes (in some cases).
23. Universal Identifiers: Includes ISBN, ISSN, DOI, barcodes, and SKUs.
24. Region-Specific Sensitive Banking & Identity Data: Includes Saudi IBAN patterns, national ID formats, and regional payment identifiers.

PII (Personally Identifiable Information): Any data that can be used to identify a specific

individual, such as full name, national ID, phone number, email address, or date of birth.

API Key: A unique secret code used to authenticate access to an external service or application programming interface.

Risk Level: A classification assigned to detected sensitive data indicating its potential impact: Low, Medium, or High.

High: includes the sensitive data from the following categories

1. Authentication & Secrets
2. Financial & Banking
3. Personal identity
4. Cryptography & Certificates
5. Developer Secrets

Medium: includes the sensitive data from the following categories

1. Contact Information
2. Network & System Data
3. Web Links & Identifiers
4. Location Data
5. Legal & Government Data
6. Vehicle Data
7. Corporate & Document Identifiers
8. File & Path Data
9. Digital Fingerprints & Technical Identifiers
10. Shipping & Logistics Data
11. Dates & Time Data
12. Logs & Headers Data
13. Numeric Sensitive Data
14. Education-Related Identifiers

Low: includes the sensitive data from the Keyword-Based Sensitive Patterns category.

Prompt: The text input entered by a user into an AI platform before submission.

Detection Engine: The core component of Hares AI responsible for analyzing user prompts and identifying sensitive data patterns.

Admin Dashboard: A web-based interface accessible by organization administrators to monitor AI usage, review logs, and configure security policies.

Policy: A set of rules configured by an administrator that defines what types/categories of sensitive data should trigger a warning or block.

Anonymized Log: A record of a detected violation that excludes raw sensitive content, storing only metadata such as risk level, timestamp, and data category.

Local Processing: The analysis of user input performed entirely on the user's device without transmitting data to external servers.

Placeholder Tag: A category-specific label (e.g., [API_KEY], [PHONE], [EMAIL]) used to replace detected sensitive data in the user's prompt before any external transmission.

System Admin: The highest-level actor with unrestricted access to system configuration, security logs, and user account creation.

Department Admin: A privileged actor with administrative rights restricted to a specific department's data and sub-users.

User: A standard actor authorized to perform day-to-day functional tasks (e.g., using AI platforms and look at the user dashboard) within their assigned department.

Administrators: branches for both system admins and department admins.

6.1 Functional Requirements

6.1.1 User Account Management

6.1.1.1 The system shall allow new users to register by providing their full name, email, and password.

6.1.1.2 The system shall allow users to log in using their email and password.

6.1.1.3 The system shall allow users to log out from their account.

6.1.1.4 The system shall allow users to reset their passwords.

6.1.1.5 The system shall assign role-based access levels (System Admin, Department Admin, User).

6.1.2 Browser Extension & Input Interception

6.1.2.1 The system shall provide a browser extension that integrates with web-based AI platforms.

6.1.2.2 The extension shall capture the user's prompt text before it is submitted to the AI platform for analysis.

6.1.2.3 The extension shall remain active across all supported AI platforms without requiring manual activation.

6.1.3 Sensitive Data Detection

6.1.3.1 The system shall analyze the captured user prompt to detect sensitive information.

6.1.3.2 The system shall classify detected content into risk levels: Low, Medium, and High.

6.1.3.3 The system shall mask detected sensitive data before any further processing or transmission.

6.1.4 Alert & Prevention

6.1.4.1 The system shall notify the user within 200 milliseconds via a User Interface alert when sensitive data is detected.

6.1.4.2 The system shall block prompt submission when the detected risk is High.

6.1.4.3 The system shall require the user to remove the sensitive data before resubmitting when the risk level is High.

6.1.4.4 The system shall display a warning when the detected risk level is Medium.

6.1.4.5 The system shall require explicit user confirmation before allowing prompt submission when the risk level is Medium.

6.1.4.6 The system shall display a non-blocking informational alert when the risk level is classified as Low.

6.1.4.7 The system shall display the prompt to the user for review when sensitive data is detected.

6.1.4.8 The system shall allow the user to edit the prompt before resubmitting.

6.1.5 Logging & Audit

6.1.5.1 The system shall record the risk level for each analyzed user prompt.

6.1.5.2 The system shall store logs of detected policy violations, including the user's name, date, and time.

6.1.5.3 The system shall allow administrators to review activity logs.

6.1.5.4 The system shall allow administrators to export activity logs in CSV format.

6.1.6 System Admin Dashboard

6.1.6.1 The system shall provide an admin dashboard for monitoring AI usage across the

organization.

6.1.6.2 The dashboard shall provide the system admin a graph illustrating the most used AI platforms in each department.

6.1.6.3 The dashboard shall provide the system admin a graph illustrating the frequently leaked sensitive data in each department.

6.1.6.4 The dashboard shall allow the system admin to deactivate a user account within the organization.

6.1.6.5 The dashboard shall allow the system admin to change a user account department within the organization.

6.1.6.6 The dashboard shall allow the system admin to assign a role to a user account (Department Admin/User).

6.1.6.7 The dashboard shall provide the system admin with a summary of risks across all departments.

6.1.7 Department Admin Dashboard

6.1.7.1 The system shall provide a Department Admin Dashboard for monitoring AI usage across the department.

6.1.7.2 The dashboard shall allow the department admin to deactivate a user account within the department.

6.1.7.3 The dashboard shall provide the department admin with a summary of risks in the department.

6.1.7.4 The dashboard shall provide the department admin a graph illustrating the most used AI platforms in their department.

6.1.7.5 The dashboard shall provide the department admin a graph illustrating the frequently leaked sensitive data category in each department.

6.1.8 User Dashboard

6.1.8.1 The system shall provide a dashboard for users to monitor their AI usage.

6.1.8.2 The dashboard shall display the user's detected risk activity.

6.1.8.3 The dashboard shall display the category of sensitive data that has been leaked.

6.1.8.4 The system shall show users a list of their own blocked messages only.

6.1.9 Privacy & Local Processing

6.1.9.1 The system shall store logs without including the user's raw prompt content, replacing any sensitive fields with placeholder tags.

6.1.9.2 The system shall not transmit raw user input to any external server.

6.1.9.3 The system shall mask all sensitive data in a prompt before transmitting any portion of it to an external classification service.

6.1.10 Prompt Assistance & Rewriting

6.1.10.1 The system shall display at least one recommended revision of the prompt that complies with the active organizational policy when sensitive data is detected.

6.1.10.2 The system shall automatically generate a rewritten version of the prompt with sensitive data removed upon detection.

6.1.10.3 The system shall suggest a restructured version of the user's prompt that includes the following elements: a defined role, context, task, output format, and tone.

6.2 Non-Functional Requirements

6.2.1 Performance

6.2.1.1 The system shall complete local Regex-based detection and return a risk classification within 200 milliseconds.

6.2.1.2 In cases requiring secondary analysis via the Gemini API, the total detection process shall not exceed 1.2 Seconds.

6.2.1.3 The browser extension shall not consume more than 100MB of RAM during normal operation.

6.2.1.4 The admin dashboard shall load within 3 seconds under normal network conditions.

The system shall generate and display the compliant prompt revision within 1.5 seconds after a BLOCK or WARN enforcement action is triggered.

6.2.2 Security

6.2.2.1 All communication between the extension and the backend shall be encrypted using HTTPS.

6.2.2.2 The system shall not transmit raw sensitive data to any external server.

6.2.2.3 The system shall enforce role-based access control to restrict admin features to authorized users only.

6.2.2.4 User passwords shall be stored using a secure hashing algorithm (e.g., bcrypt).

6.2.3 Usability

6.2.3.1 The extension shall require no more than 3 steps to install and activate.

6.2.3.2 The warning message shall clearly identify the type and risk level of detected sensitive data.

6.2.3.3 The admin dashboard shall be accessible from any modern web browser without additional software installation.

6.2.4 Reliability

6.2.4.1 The system shall achieve a minimum detection accuracy of 90% for structured sensitive data patterns such as API keys, emails, and phone numbers.

6.2.4.2 The false positive rate shall not exceed 10%.

6.2.4.3 The extension shall continue to function using local Regex detection even when the backend is unavailable.

6.2.5 Maintainability

6.2.5.1 The detection patterns shall be updatable without requiring a full system redeployment.

6.2.5.2 The system shall be built using a modular architecture to allow independent updates to each component.

6.2.6 Compatibility

6.2.6.1 The extension shall be compatible with Google Chrome version 100 and above.

6.2.6.2 The extension shall be compatible with Chromium-based browsers including Microsoft Edge.

6.2.6.3 The system shall support integration with ChatGPT, Google Gemini, and Claude at launch.

7. Problem Complexity

The development of Hares AI involves several dimensions of complexity that go beyond standard software engineering challenges. The following points highlight the key areas of complexity in this project.

7.1 Technical Complexity

The core challenge of Hares AI lies in accurately detecting sensitive data in real-time within a browser extension environment. Unlike traditional applications, browser extensions operate under strict security sandboxing rules defined by Manifest V3, which limits access to system resources and restricts how scripts interact with web pages. Intercepting user input across multiple AI platforms each with a different DOM structure requires platform-specific implementations that must be maintained as these platforms evolve.

7.2 No Obvious Solution

There is no single established approach for real-time, privacy-preserving sensitive data detection inside a browser extension. Existing solutions either rely on cloud-based processing which compromises privacy or are limited to a single platform. Hares AI must balance detection accuracy, processing speed, and privacy simultaneously, a combination that has no off-the-shelf solution.

7.3 Multiple Disciplines Involved

The project spans multiple disciplines including cybersecurity, natural language processing, web development, and software architecture. Each component, the extension, the backend, and the admin dashboard requires different technical expertise and must be integrated seamlessly.

7.4 Significant Consequences

A false negative failing to detect sensitive data could result in a real data breach. A false positive incorrectly blocking safe content could disrupt workflows and reduce user trust. Both failure modes carry meaningful consequences, raising the stakes for detection accuracy.

7.5 Wide-Ranging Technical Issues

The system must handle diverse data types (structured and unstructured), multiple languages, and constantly changing AI platform interfaces. Each of these variables introduces additional complexity that must be addressed through robust design and testing.

8. System Use-Cases

8.1 Use Cases Diagram

8.1.1 User Account Management

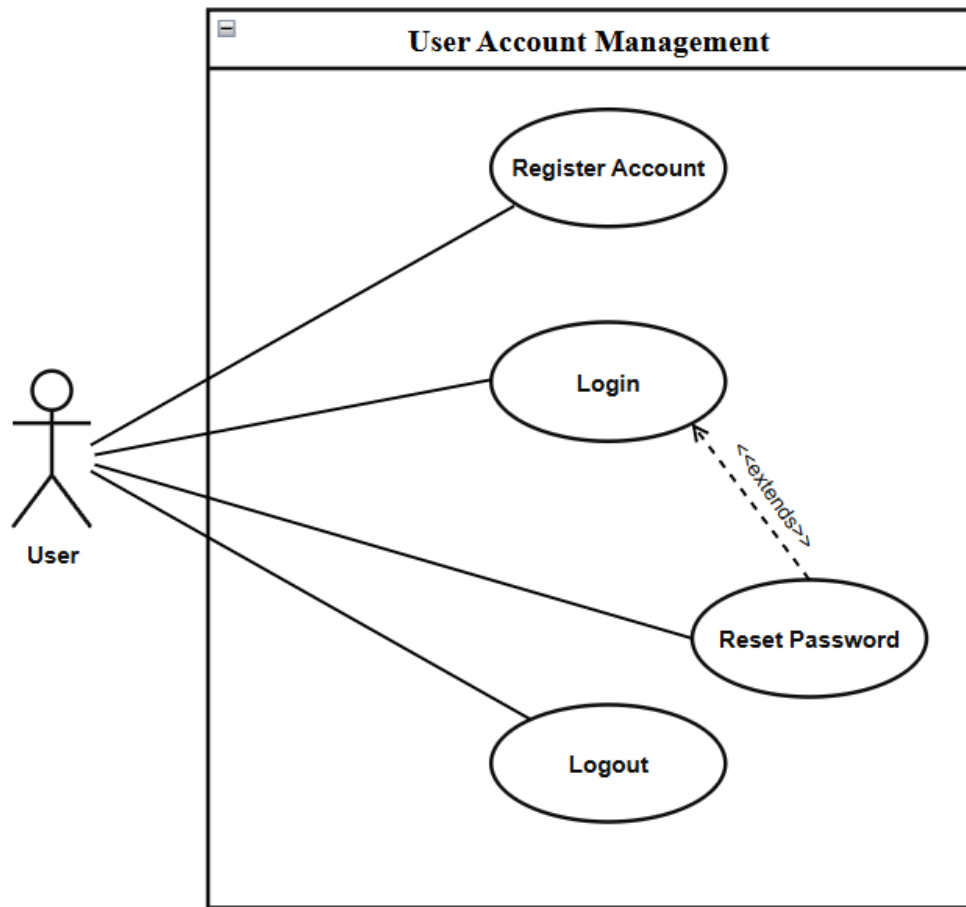


Figure 2. User Account Management.

8.1.2 Prompt Interception & Detection

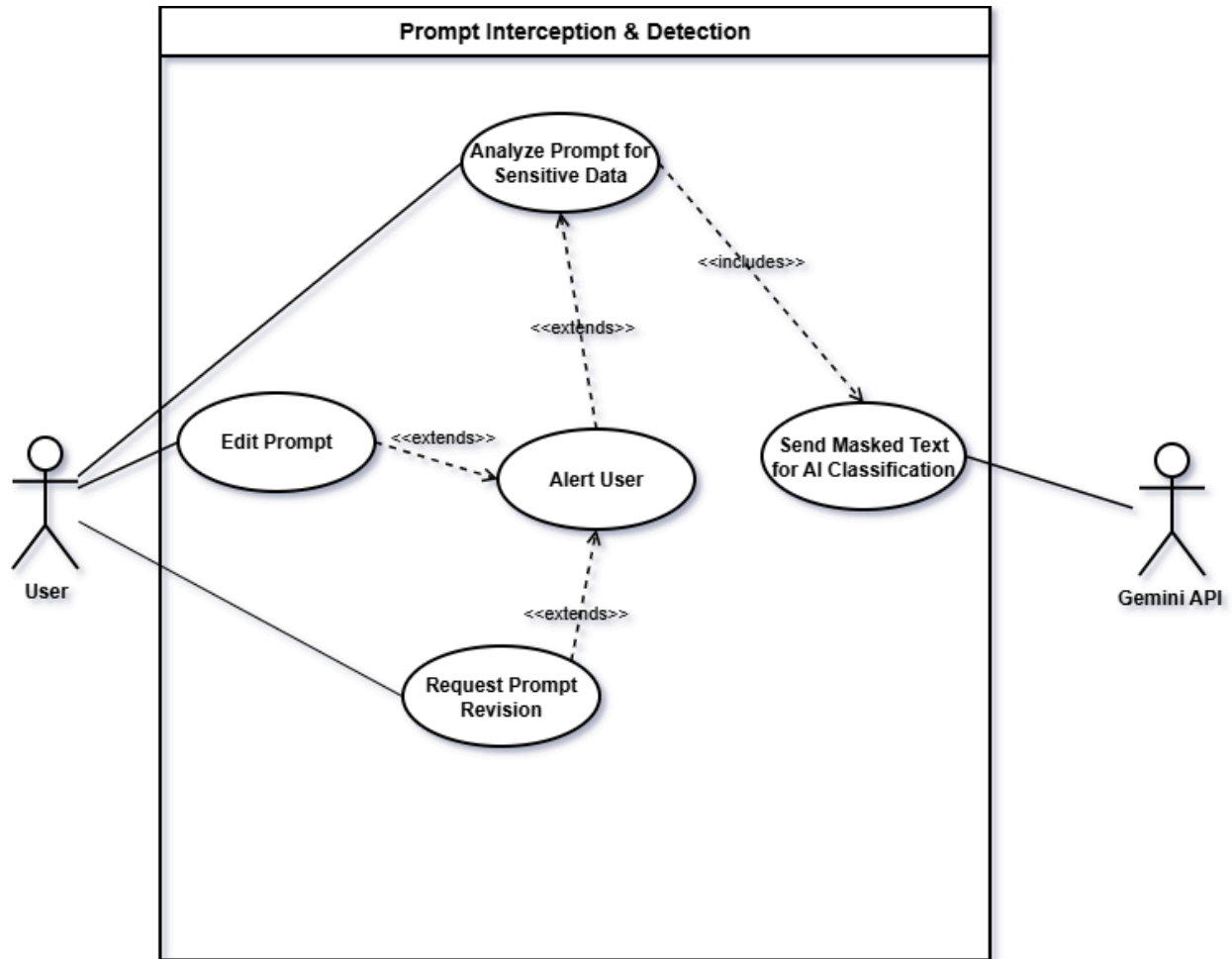


Figure 3. Prompt Interception & Detection use case.

8.1.3 Logging, Audit & Prompt Assistance

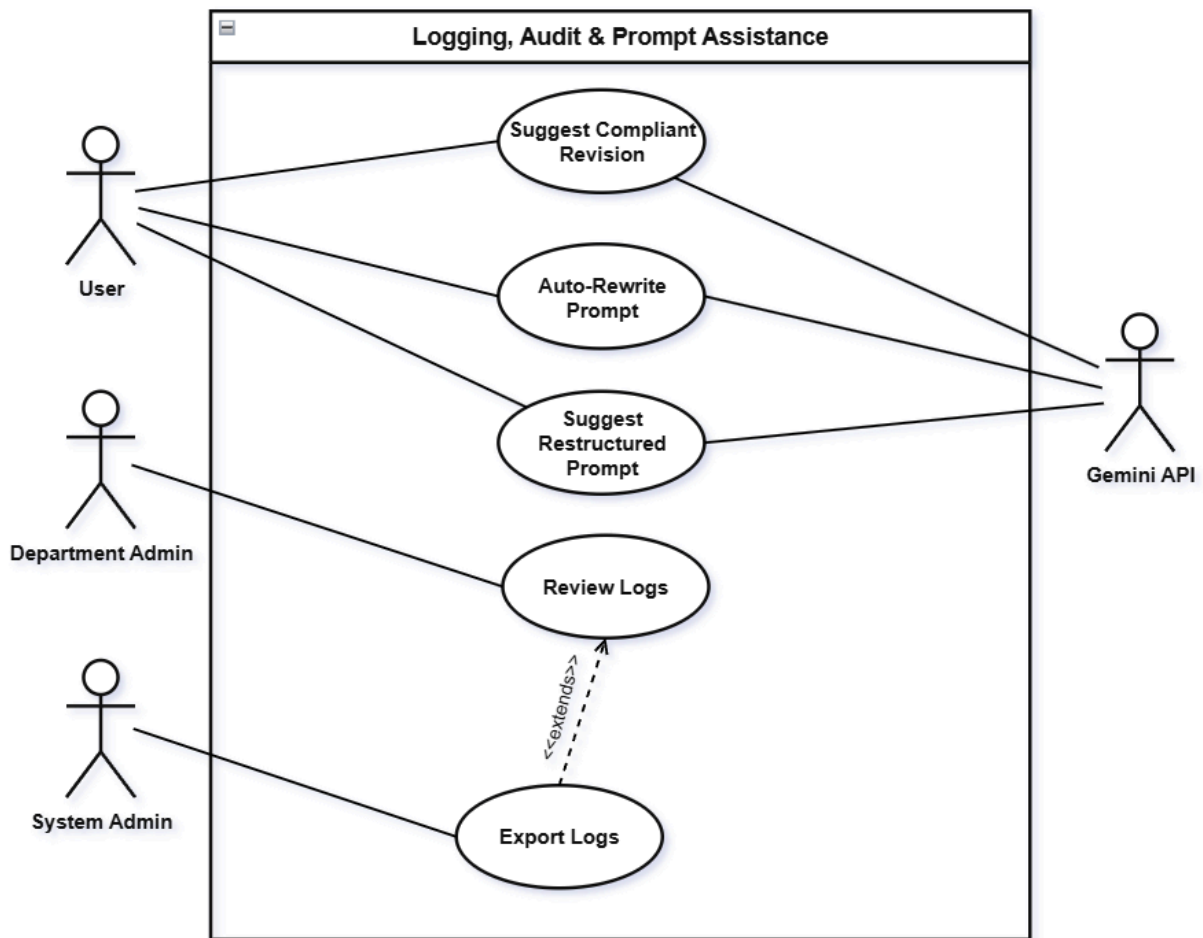


Figure 4. Logging, Audit & Prompt Assistance.

8.1.4 Dashboard Management

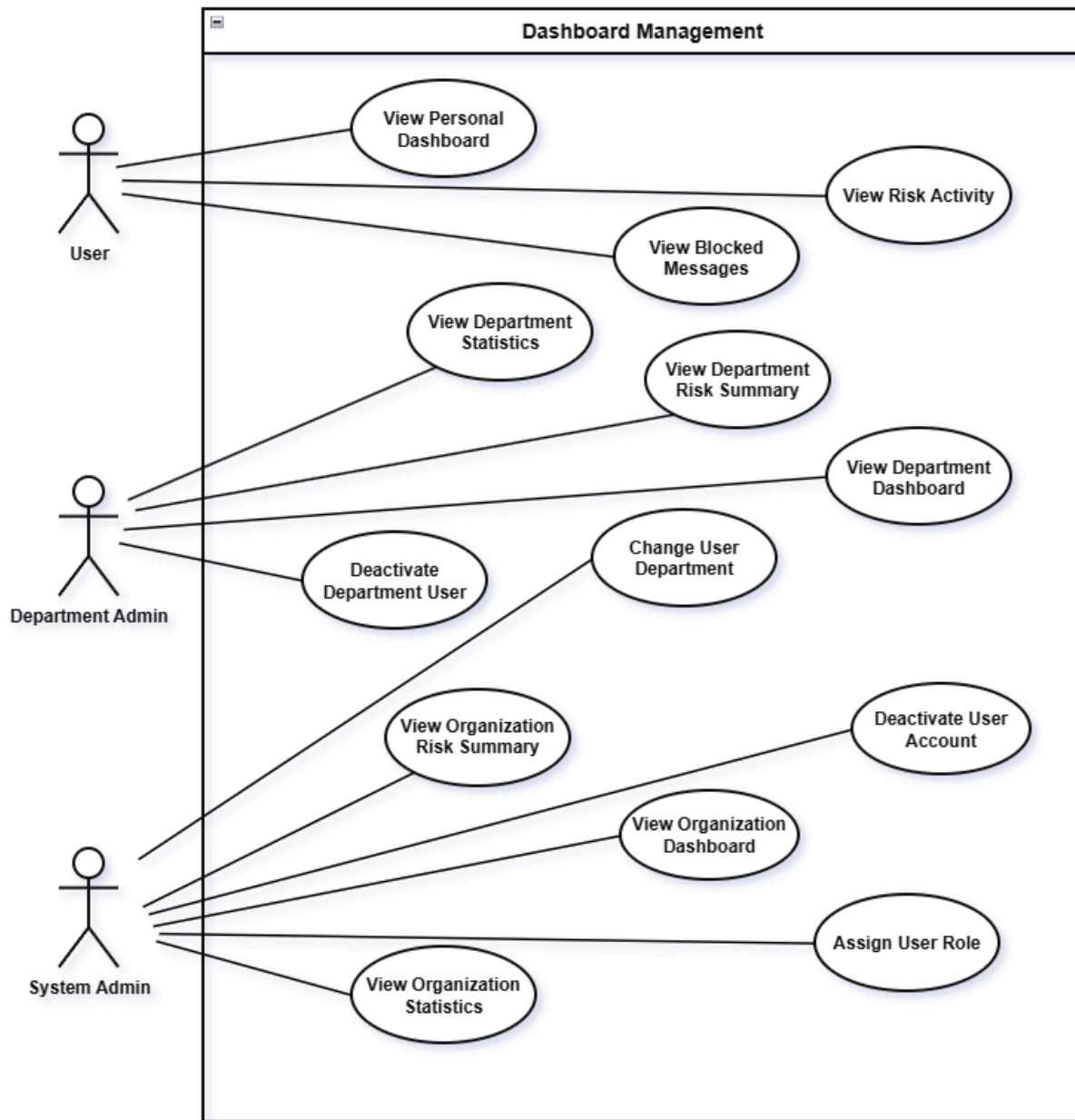


Figure 5. Dashboard Management use case.

8.2 Use Cases Description

8.2.1 Login

Use Case Name: Login	
Primary actor: User	Other actors: None

Stakeholders: Organization	
Description: A registered user authenticates into the system using their email and password to access the browser extension and dashboard features.	
Relationships: extends: Reset Password	
Input: Email address, password	
Pre-conditions: The user has a registered account in the system. The user is not currently logged in.	
Steps	
User	System
1. The user provides their email address and password.	2. The system validates that both fields are provided. 3. The system sends the credentials to Firebase Auth or verification. 4. The system receives authentication confirmation. 5. The system retrieves the user's assigned role. 6. The system grants access and redirects the user to their corresponding dashboard.
Alternative and exceptional flows: 2a.1: If any field is empty, the system displays an error message requesting the user to complete all required fields. 4a.1: If the credentials are invalid, the system displays an invalid credentials error message. 4b.1: If the account has been deactivated, the system denies access and informs the user that their account is inactive	
Post-conditions: The user is authenticated and has access to system features based on their assigned role.	

8.2.2 Edit Prompt

Use Case Name: Edit Prompt	
Primary actor: User	Other actors: None
Stakeholders: Organization	
Description: The user modifies the content of a prompt that was flagged for containing sensitive data, removing or replacing the flagged content before attempting to send it again.	
Relationships: extends: Alert User	
Input: Modified prompt text	
Pre-conditions: The system has detected sensitive data in the user's prompt. The system has displayed an alert showing the detected sensitive content. The user has chosen to edit the prompt rather than discard it.	
Steps	
User	System
1. The user selects the option to edit the prompt.	
	2. The system displays the prompt in an editable view with the detected sensitive data highlighted.
3-The user modifies the prompt content to remove or replace the flagged sensitive data.	
4. The user confirms the edits.	
	5. The system re-analyzes the modified prompt for sensitive data.
	6. The system updates the risk classification based on the new content.
Alternative and exceptional flows: 3a.1: If the user cancels the editing process, the system returns to the previous alert view without applying any changes. 5a.1: If the modified prompt still contains sensitive data, the system displays a new alert with the updated risk level. 5b.1: If the modified prompt no longer contains sensitive data, the system allows the prompt to proceed for submission. 6a.1: If the user closes the editor without confirming, the system discards the changes and retains the original flagged prompt.	

Post-conditions: The prompt content has been updated by the user, and the system has either cleared the alert or issued a new alert based on the re-analysis result.

8.2.3 Review Logs

Use Case Name: Review Logs

Organization Dashboard

Use Case Name: View Organization Dashboard	
Primary actor: System Admin	Other actors: None
Stakeholders: Organization	
Description: The System Admin views the main dashboard displaying organization-wide security statistics, AI platform usage trends, and sensitive data leak categories.	
Relationships: None	
Input: None	
Pre-conditions: The System Admin is authenticated. The System Admin has system-level administrative access.	
Steps	
User	System
1. The System Admin navigates to the organization dashboard.	2. The system retrieves aggregated data from all departments. 3. The system displays summary statistics: total intercepted prompts, total blocked prompts, total warnings issued, and percentage of high-risk detections. 4. The system displays a graph illustrating the most used AI platforms across departments. 5. The system displays a graph illustrating the most frequently leaked sensitive data categories.
Alternative and exceptional flows: 2a.1: If no activity data has been recorded, the system displays a no-data message. 2b.1: If the data retrieval fails, the system displays an error message and prompts the administrator to retry.	

Post-conditions: The System Admin has viewed the organization-wide security statistics and AI usage data.
--

8.2.5

Suggest Compliant Revision

Use Case Name: Suggest Compliant Revision	
Primary actor: User	Other actors: Gemini API
Stakeholders: Organization	
Description: The system provides the user with a recommended revision of their prompt that complies with the organization's active security policy after sensitive data has been detected.	
Relationships: None	
Input: The original prompt with detected sensitive data masked by placeholder tags	
Pre-conditions: The system has detected sensitive data in the user's prompt. The user has chosen to review the prompt. The organization's security policy is configured and active.	
Steps	
User	System
1. The user requests a compliant revision of the prompt.	2. The system retrieves the organization's active security policy and the masked version of the prompt. 3. The system sends the masked prompt to the Gemini API with instructions to generate a compliant revision. 4. The system receives the suggested revision from the Gemini API. 5. The system displays the suggested revision alongside the original prompt.

Alternative and exceptional flows:

3a.1: If the Gemini API is unavailable, the system informs the user that the suggestion service is temporarily unavailable and allows manual editing.

4a.1: If the Gemini API returns an incomplete or irrelevant suggestion, the system informs the user and allows manual editing.

6a.1: If the user accepts the revision, the system replaces the original prompt with the revised version.

6b.1: If the user discards the revision, the system returns the user to the review view with the original prompt.

Post-conditions: The user has received a suggested compliant revision and has chosen to accept, modify, or discard it.

9. Analysis Class

The Analysis Class Diagram for Hares AI is organized around five functional groups, each representing one major use case of the system. Every group follows the Boundary-Control-Entity (BCE) pattern, where boundary classes handle user interaction, control classes carry the business logic, and entity classes hold the persistent data. The User Account Management group covers authentication and account lifecycle through LoginInterface, RegistrationInterface, AccountManagementInterface, AuthController, AccountController, and the User entity. The Prompt Interception and Detection group forms the core of the system, where PromptInputInterface captures the user's input, PromptInterceptionController orchestrates the pipeline, DetectionController runs local regex scanning and risk classification, and the results are stored in the Prompt and DetectionResult entities. The Logging and Audit group provides administrators and users with access to the activity history through LogReviewInterface, UserActivityInterface, LogController, and the ActivityLog entity. The Dashboard Management group serves three role-specific dashboard interfaces organisational, departmental, and personal all driven by a single DashboardController that aggregates ActivityLog data into DashboardStatistics snapshots. Finally, the Prompt Assistance and Rewriting group allows users to receive a safe alternative for blocked prompts, managed by RevisionController and stored as a RevisedPrompt entity. Shared entities such as ActivityLog and Prompt appear across multiple groups, reflecting the system's integrated data flow from interception through logging and analytics.

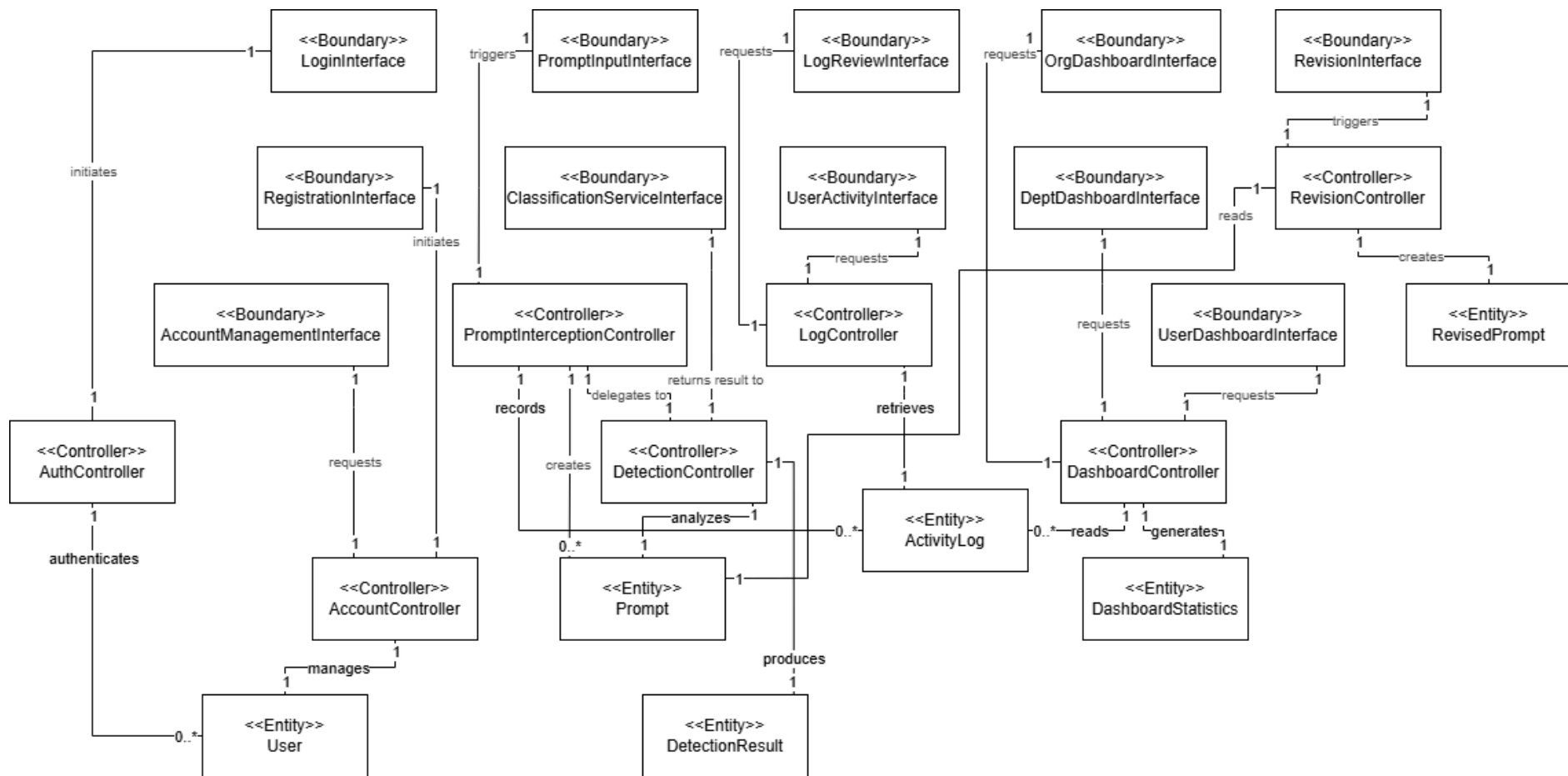


Figure 6. Hares AI Analysis Class.

10. Interaction Diagram

10.1 Login Interaction Diagram

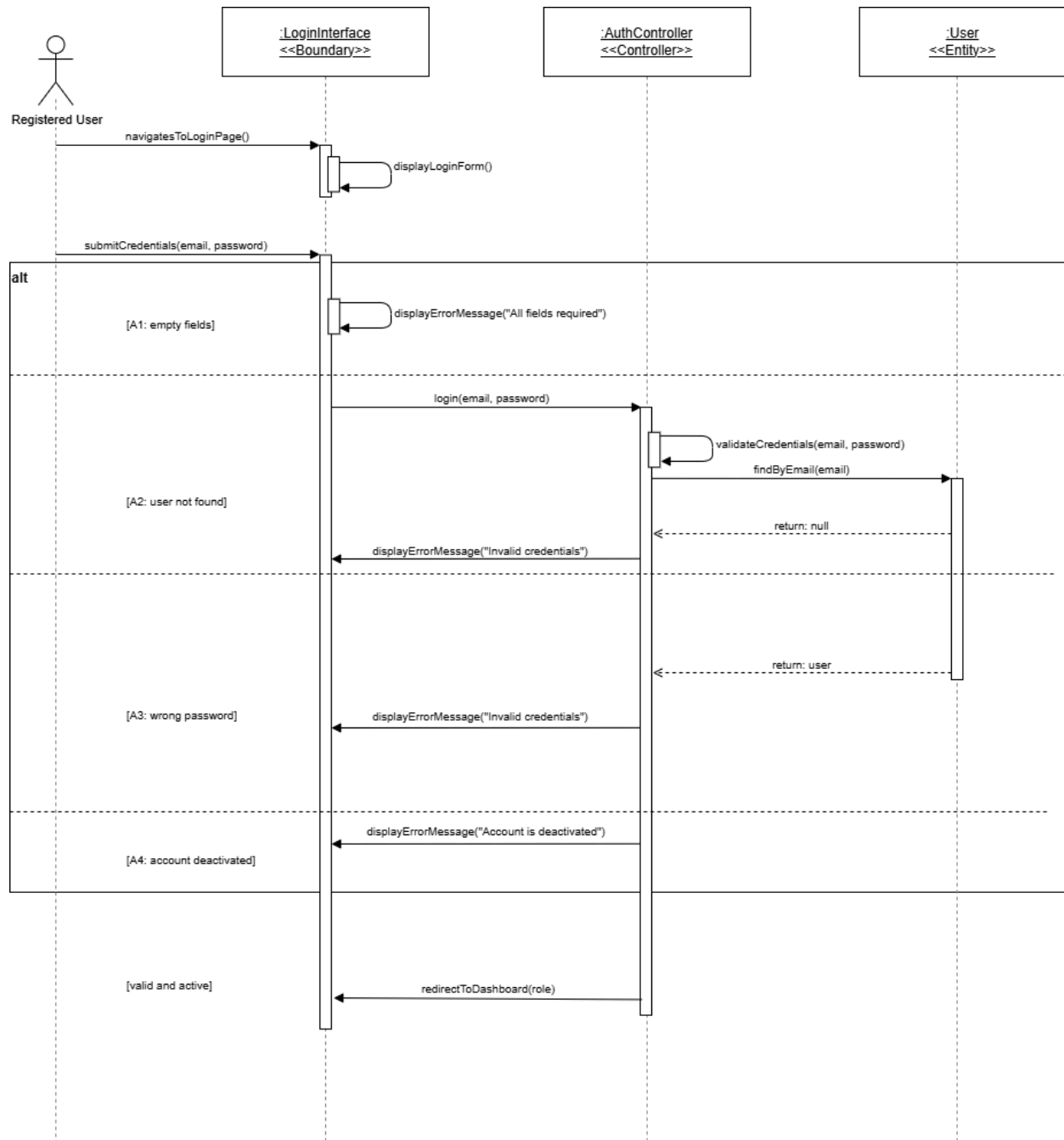


Figure 7. Login Interaction Diagram.

10.2 Registration Interaction Diagram.

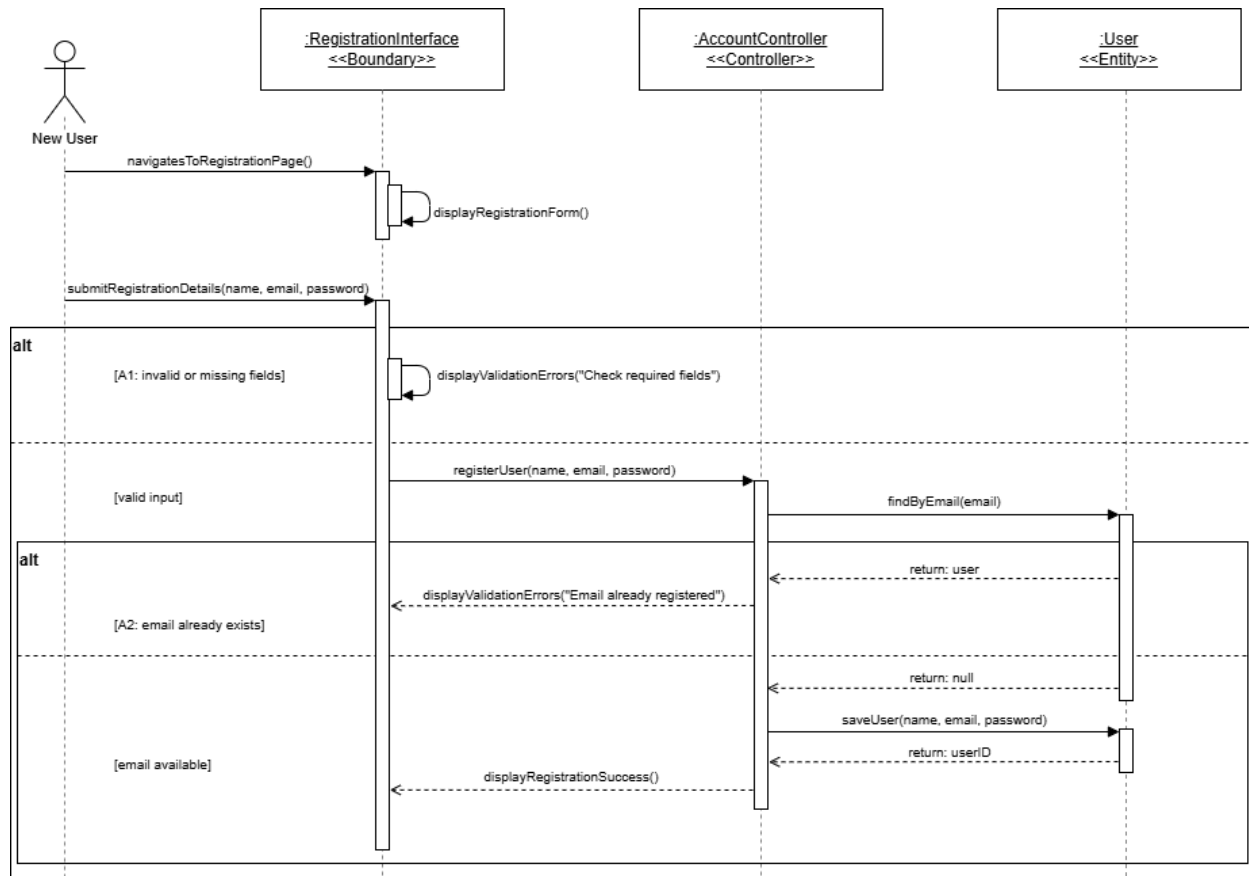


Figure 8. Registration Interaction Diagram.

10.3 Review Logs Interaction Design.

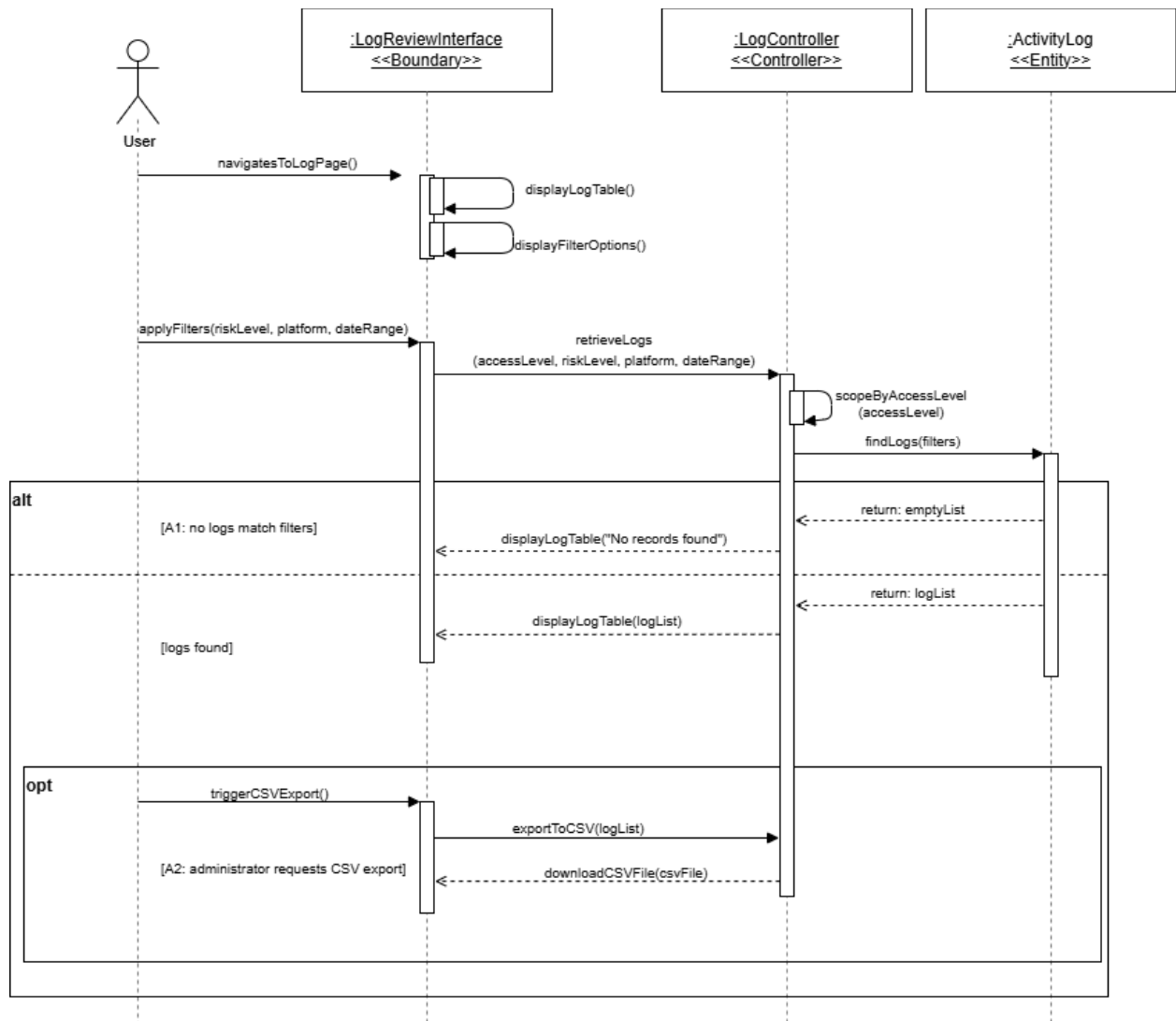


Figure 9. Review Logs Interaction Diagram.

10.4 Organization Dashboard.

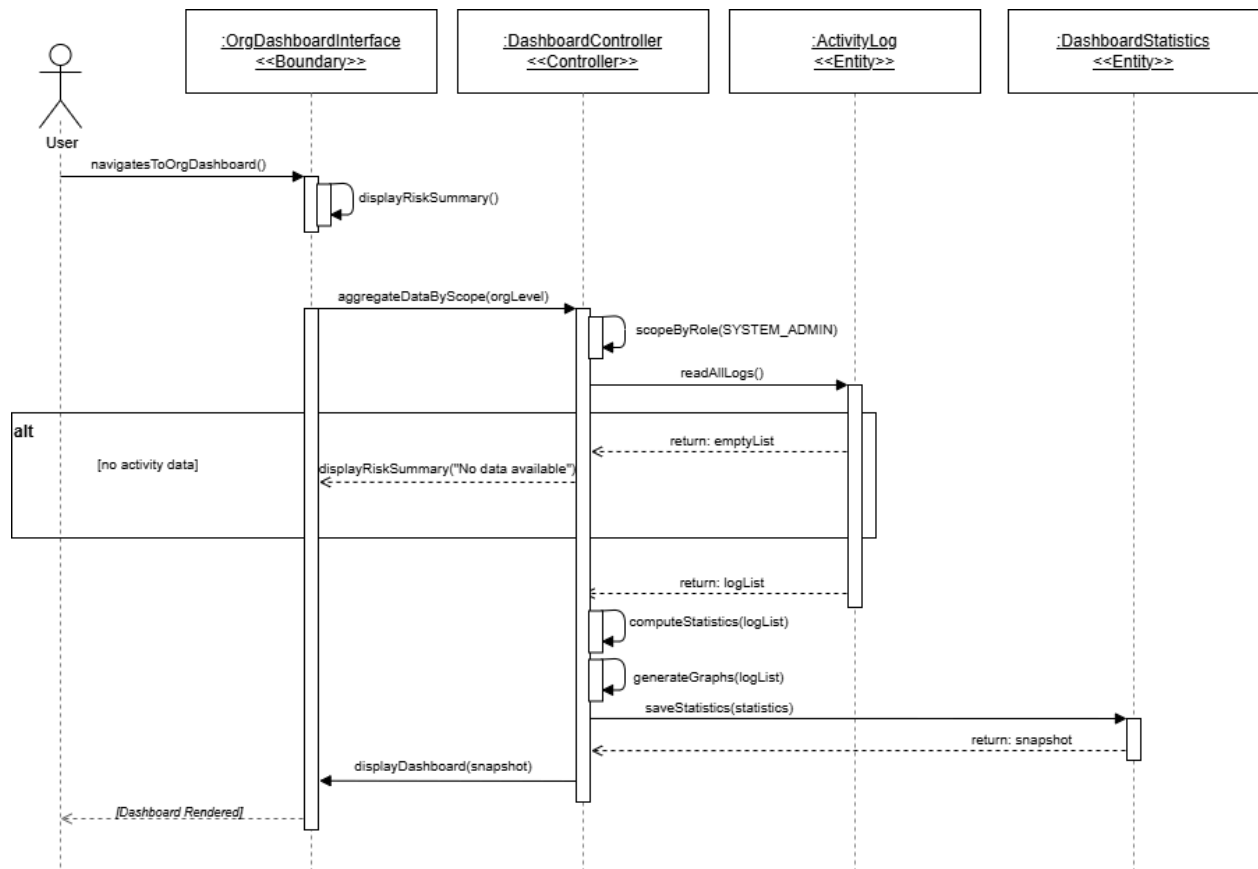


Figure 10. Organization Dashboard Interaction Diagram.

10.5 Revision Prompt Interaction Diagram.

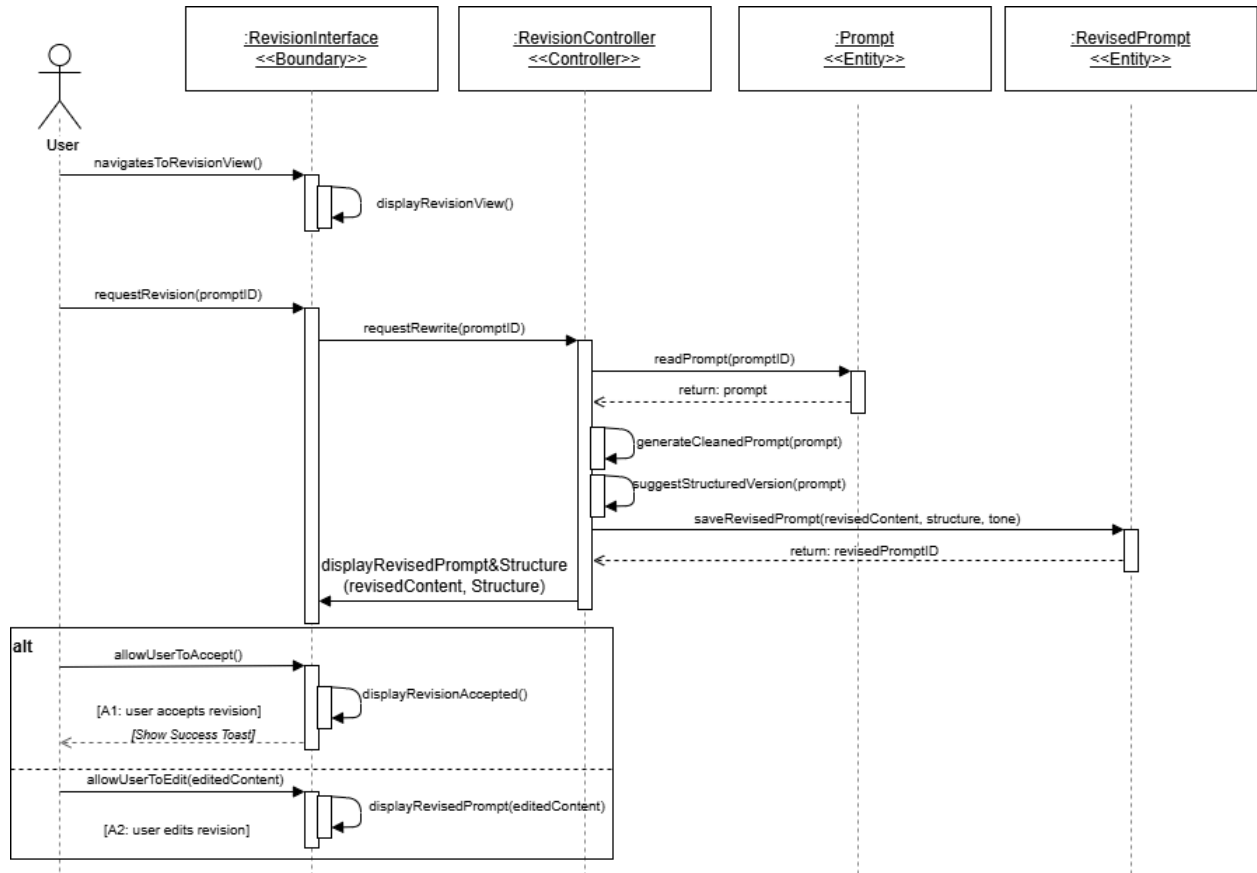


Figure 11. Revision Prompt Interact.

11. Design Class

Classes:

LoginService: Handles all authentication related operations. It is responsible for verifying user credentials, managing login and logout sessions, triggering password reset flows, retrieving the user's role after successful authentication, and directing the UI to display the appropriate form or error message.

AccountService: Manages user account lifecycle. It is responsible for registering new users, assigning or changing roles, deactivating accounts, moving users between departments, and presenting the user list to administrators.

PromptService: The core processing engine of the system. It intercepts the user's prompt before submission, runs local regex detection, classifies the risk level, enforces the organisation's policy by deciding whether to block, warn, or allow, masks sensitive data, optionally requests secondary classification from the Gemini API, and records the result as a log entry.

LogService: Handles retrieval and export of activity logs. It scopes the log view based on the requesting user's role, applies filters, returns matching log entries, and can export them to CSV.

DashboardService: Aggregates activity data and produces statistics for visualisation. It scopes data by role (system-wide, department-wide, or personal), computes summary statistics, and generates the graphs and summaries shown on the dashboard.

RevisionService: Generates safe, restructured alternatives for blocked or warned prompts. It takes the original prompt, produces a cleaned version with sensitive data removed, optionally suggests a more structured wording, and presents both options to the user for acceptance or further editing.

User: Represents a registered user of the system. It stores identity and account data (name, email, hashed password, department, status) and the user's assigned role. It is the central entity that most services and logs reference.

Prompt: Represents a single intercepted prompt submission. It stores the original text, the masked version produced by the detection engine, the platform on which it was typed, and the timestamp of interception.

DetectionResult: Represents the outcome of the detection pipeline for one prompt. It stores which data category was detected, the assigned risk level, and the action that was taken. It is produced by PromptService and linked directly to the Prompt it analysed.

ActivityLog: A persistent audit record of a prompt interception event. It stores the user identifier, the data category involved, the risk level, the action taken, the platform, and the timestamp. It is the primary data source for both LogService and DashboardService.

DashboardStatistics: A computed snapshot of aggregated activity data. It holds counters (total intercepted, blocked, warnings), percentages (high-risk share), and categorical summaries (most-used platform, top leaked category). It is produced on demand by DashboardService from ActivityLog data.

RevisedPrompt: Represents the output of the revision pipeline for a specific prompt. It stores the cleaned content, the suggested structured version, and the tone label. It is produced by RevisionService and linked back to the original Prompt.

RiskLevel: An enumeration defining the four possible risk classifications: HIGH, MEDIUM, LOW, and NONE. Used by PromptService to label detection results and by ActivityLog and DetectionResult to record what level was assigned.

ActionType: An enumeration defining the three possible enforcement outcomes: BLOCKED, WARNED, and ALLOWED. Used by PromptService to communicate its decision and stored in DetectionResult and ActivityLog.

UserRole: An enumeration defining the three access levels in the system: SYSTEM_ADMIN, DEPARTMENT_ADMIN, and USER. Used by LoginService and AccountService to assign and check permissions, and stored in the User entity.

Platform: An enumeration of the supported AI platforms: CHATGPT, GEMINI, and CLAUDE. Used by PromptService to tag where the prompt originated, and stored in Prompt, ActivityLog,

and DashboardStatistics.

Association:

- 1:** LoginService manages the authentication of User accounts. It reads and verifies user records to validate credentials and retrieve role information.
- 2:** AccountService creates, deactivates, and modifies User records. Every registration, role change, or department transfer is applied to a User entity through this service.
- 3:** PromptService creates and processes Prompt records. Each intercepted submission becomes a Prompt instance that PromptService reads and acts upon.
- 4:** For every processed Prompt, PromptService produces a DetectionResult that records what was found, the risk level assigned, and the action taken.
- 5:** After each prompt is processed, PromptService writes an ActivityLog entry capturing the full audit record of that interception event.
- 6:** LogService reads and filters ActivityLog entries to serve retrieval requests from administrators and users. It has no write access it only queries existing records.
- 7:** DashboardService aggregates ActivityLog entries to produce statistical summaries. It reads log data scoped by the requesting user's role.
- 8:** DashboardService computes and produces DashboardStatistics objects from the aggregated log data. These statistics are then used to populate the dashboard views.
- 9:** RevisionService reads the original Prompt content in order to generate a cleaned and restructured alternative. It accesses the Prompt's original and masked text as its input.
- 10:** RevisionService produces RevisedPrompt records containing the cleaned version and structured suggestion generated for a given Prompt.
- 11:** Every Prompt is submitted by exactly one User. A User may have zero or many associated Prompts over time.
- 12:** Each DetectionResult is tied to exactly one Prompt that it analysed. There is a one-to-one correspondence between a detection outcome and the prompt that triggered it.
- 13:** Each ActivityLog entry is associated with the User whose prompt triggered the event. A single User can accumulate many log entries over time.
- 14:** Each RevisedPrompt is derived from exactly one original Prompt. It stores the cleaned and restructured alternative produced for that specific interception event.
- 15:** DashboardStatistics is a derived object computed from a set of ActivityLog records. It does

not own the log data; it is a read-only aggregate that reflects the state of the logs at the time it was generated.

16: LoginService uses the UserRole enumeration to retrieve and interpret the role of an authenticated user, determining which dashboard to redirect them to.

17: AccountService uses UserRole when assigning or changing the role of a user account.

18: PromptService uses RiskLevel to label the output of the detection pipeline and to drive the policy enforcement decision.

19: PromptService uses ActionType to record and communicate whether a prompt was blocked, warned, or allowed.

20: PromptService uses Platform to tag each intercepted prompt with the AI service on which it was typed.

21: LogService uses UserRole to scope the log query a SYSTEM_ADMIN sees all logs, a DEPARTMENT_ADMIN sees their department's logs, and a USER sees only their own.

22: DashboardService uses UserRole to determine the scope of data aggregation presented on the dashboard.

23: DashboardService uses Platform when generating the platform usage graph that shows activity distribution across ChatGPT, Gemini, and Claude.

24: The User entity stores its access level as a UserRole value. The get Role method returns this typed value to any service that needs to check permissions.

25: The Prompt entity stores which AI platform it was captured from as a Platform value.

26: DetectionResult stores the risk classification assigned to the analysed prompt as a RiskLevel value.

27: DetectionResult stores the enforcement decision taken for the prompt as an ActionType value.

28: ActivityLog stores the risk level of the logged event as a RiskLevel value for filtering and statistical queries.

29: ActivityLog stores the action taken during the logged event as an ActionType value.

30: ActivityLog stores the platform on which the intercepted prompt occurred as a Platform value, enabling platform-based filtering and dashboard breakdowns.

31: DashboardStatistics stores the most-used platform as a Platform value within its snapshot

summary.

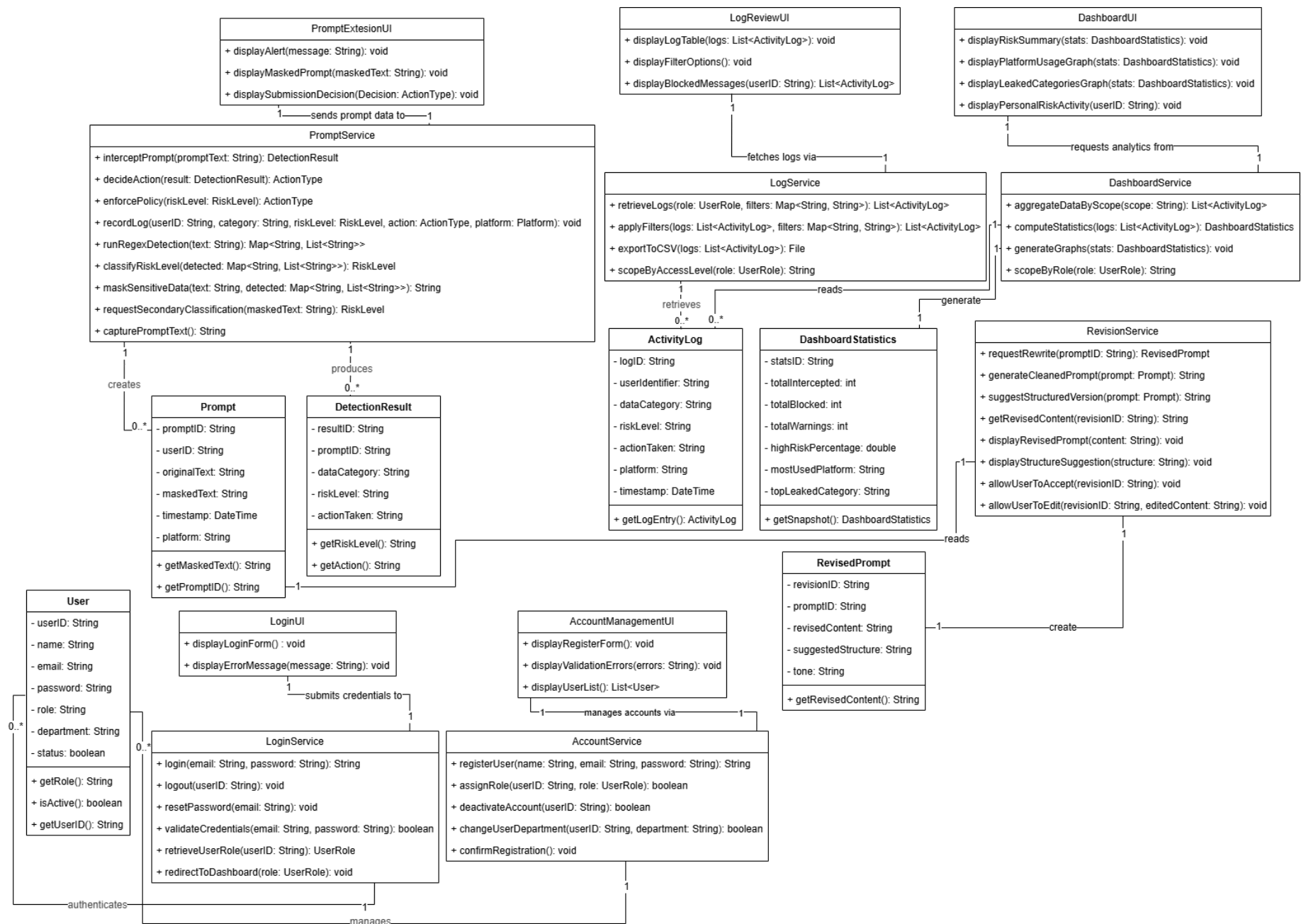


Figure 12. Hares AI Class Diagram.

To see The Class Diagram clear way Click [here](#)

12. System Architecture

Based on the functional and non-functional requirements of Hares AI, we selected the **Layered Architecture** as the primary architectural style for the system. In this style, the system is divided into distinct horizontal layers, each with a clearly defined responsibility. Each layer only communicates with the layer immediately below it, which enforces strict separation of concerns and prevents components from becoming tightly coupled.

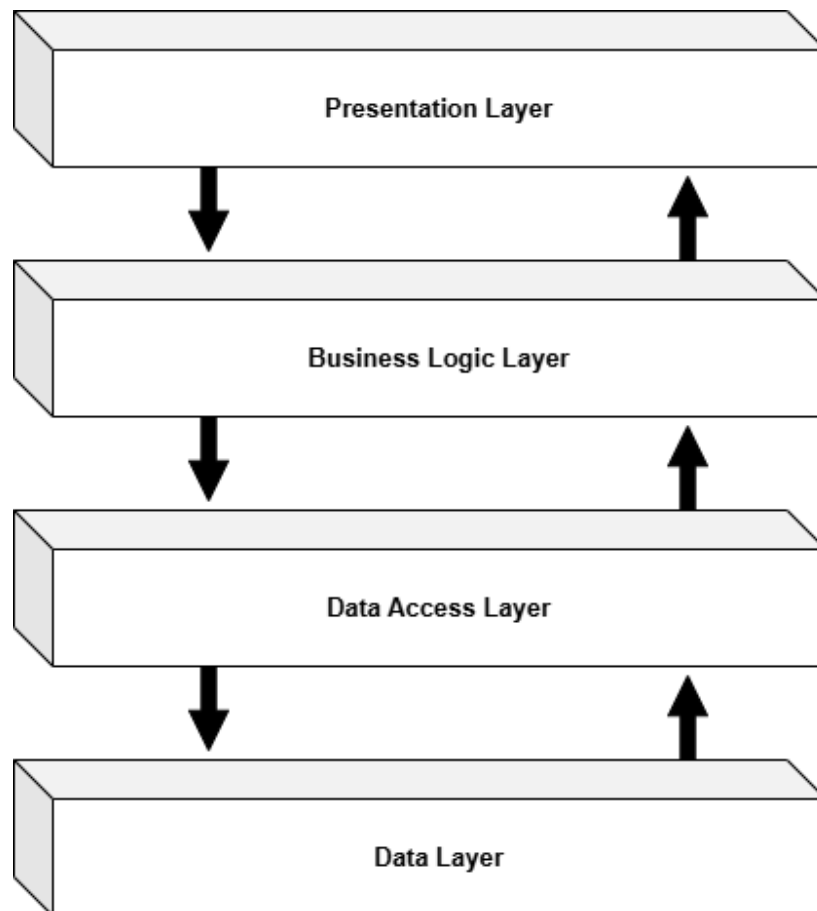


Figure 13. System Architecture.

The Hares AI system is organized into four layers:

Presentation Layer This layer contains all components that interact directly with the user.

It includes the Content Script UI (the alert overlays and masked-prompt views injected into AI platforms such as ChatGPT, Gemini, and Claude), the Extension Popup (allowing the user to view system status and configure personal settings), and the Admin Web Dashboard (used by

System Administrators and Department Administrators to view logs, statistics, and manage user accounts).

This layer receives input from users and renders output, but contains no business logic.

Business Logic Layer This is the core of the system. It contains the Prompt Interceptor (which captures the user's input before submission), the Regex Detection Engine (which locally scans the text for sensitive data patterns across all risk categories), the Risk Classifier (which assigns a risk level based on detection results and optionally invokes secondary AI classification), the Policy Enforcer (which decides whether to block, warn, or allow the prompt based on organisational policy), the Revision Generator (which produces a cleaned and structured alternative for the user), and the Log Manager (which constructs and routes activity log entries). None of the components in this layer depend on specific data storage technologies they operate through abstractions provided by the layer below.

Data Access Layer This layer acts as the bridge between business logic and raw storage. It contains the Chrome Storage API Client (for reading and writing configuration and cached data within the browser), the Gemini API Client (for securely sending masked text to Google's Gemini API when secondary classification is required), and the Firebase Client (for authenticating users and synchronising logs and user configurations with Firestore).

Data Layer This layer holds the physical data stores. It contains Chrome Local Storage (for extension settings, detection policies, and temporary prompt caches) and Firebase Firestore (for persistent storage of user accounts, activity logs, and organisational configurations).

This architecture was chosen for Hares AI for the following reasons. First, the separation between the Presentation Layer and the Business Logic Layer means that the extension's UI components can be updated or restyled without any risk of breaking the detection pipeline. Second, isolating the Regex Detection Engine and Risk Classifier in the Business Logic Layer means that detection patterns and risk thresholds can be tuned without touching the storage or UI code. Third, the Data Access Layer shields the business logic from knowing whether data is stored in Chrome Storage or Firebase, a critical flexibility given that the extension operates both locally and with optional cloud sync. Fourth, the Layered style naturally supports the system's privacy-first requirement: raw sensitive text never leaves the Business Logic Layer; only masked text is passed down to the Data Access Layer when an external API call is needed.

Supporting Pattern Event-Driven Architecture

The Layered Architecture is complemented by an **Event-Driven** pattern at the browser extension level. Chrome's Manifest V3 extension model is inherently asynchronous: the Content Script runs inside the AI platform's tab, the Background Service Worker runs in a separate isolated context, and the Popup runs in its own window. These components cannot share memory

directly; they communicate exclusively through Chrome's message-passing API.

This event-driven communication is used as follows. When the Content Script detects that the user has initiated a prompt submission, it fires a message carrying the prompt text to the Background Service Worker. The Service Worker runs the full detection pipeline (Regex → Risk Classification → Policy Decision) and fires a response message back to the Content Script specifying the action to take (block, warn with masked version, or allow). This design keeps the heavy processing off the UI thread and ensures that the extension remains responsive even during complex regex evaluation or API calls.

Alternative Architectural Styles Considered

Model-View-Controller (MVC)

MVC is one of the most widely adopted architectural patterns for interactive applications. It separates the system into a Model (data and business rules), a View (user interface), and a Controller (intermediary that handles user input and updates both Model and View). We considered MVC because it is familiar, well-documented, and maps cleanly onto many web applications.

However, MVC was not appropriate for Hares AI. MVC assumes a single shared runtime where the View, Controller, and Model can hold references to one another. A Chrome extension does not have a single runtime: the Content Script, the Background Service Worker, and the Popup each operate in isolated JavaScript environments with no shared memory. Attempting to apply MVC would require forcing an artificial Controller onto one of these contexts, creating unnatural coupling between components that cannot directly communicate. Furthermore, MVC does not enforce the strict data-flow isolation that Hares AI requires, in particular, the guarantee that raw sensitive text never reaches the storage or API layers. The Layered Architecture makes this boundary explicit and structural, whereas MVC would require it to be enforced only by developer convention.

Client-Server Architecture

The traditional Client-Server model treats the system as a thin client that sends requests to a centralized server, where all business logic and data storage reside. We considered this approach because Hares AI does have a server-side component (Firebase for user management and log storage) and an external API dependency (Gemini for secondary classification).

Client-Server was rejected for two fundamental reasons. First, it directly contradicts the system's most critical non-functional requirement: privacy. In a client-server model, the user's prompt text would be transmitted to a server for processing. Even if that server were trusted, the mere transmission of potentially sensitive data over the network introduces a point of exposure. Hares AI's core value proposition is that sensitive data is detected and masked locally, on the user's machine, before anything is sent anywhere. Second, the client-server introduces a single point of failure. If the server is unavailable, the extension cannot function. Hares AI is designed to continue operating offline using its local Regex Detection Engine, with the Gemini API call being an optional enhancement rather than a hard dependency. The Layered Architecture supports this graceful degradation naturally.

Microservices Architecture

Microservices decompose the system into a collection of small, independently deployable services that communicate over a network (typically via REST or message queues). We considered this approach because Hares AI has several logically distinct capabilities detection, classification, logging, revision generation that could theoretically be separated into services.

Microservices were rejected because the operational overhead is disproportionate to the project's scale and team size. Each microservice requires its own deployment pipeline, health monitoring, API versioning, and inter-service security. More critically, all network-based inter-service communication would reintroduce the same privacy exposure risk identified above: prompt text (even masked) would transit a network boundary at every pipeline step. The Layered Architecture achieves the same logical separation of concerns within a single process, without the infrastructure complexity or the privacy risk.

Implementation Constraints Affecting Architecture

Several constraints shaped the architectural decisions described above:

Chrome Manifest V3 Restrictions Manifest V3 prohibits persistent background pages and restricts the use of remote code execution. Background processing must be implemented as a Service Worker, which can be terminated by the browser at any time and must re-initialise on demand. This forces all states that must persist between events to be stored explicitly in Chrome Storage, rather than in module-level variables.

No Raw Data Transmission The system's privacy requirements prohibit transmitting unmasked prompt text to any external service. This constraint is enforced architecturally: the Gemini API Client in the Data Access Layer only accepts pre-masked text from the Risk Classifier. The masking step is structurally required before the data reaches the layer that has network access.

Cross-Browser Portability Limitation The current architecture relies on Chrome-specific APIs. This limits deployment to Chromium-based browsers (Chrome, Edge, Brave). Extending to Firefox would require replacing the Data Access Layer's Chrome Storage API Client with the WebExtensions-compatible API equivalents.

Offline-First Detection The Regex Detection Engine must function without network access. The architecture satisfies this by placing all local detection inside the Business Logic Layer, which has no dependency on the Data Access Layer's network clients. The Gemini API Client is invoked only when the Risk Classifier determines that the regex result is ambiguous, making the external call an optional enhancement rather than a required path.

13. User Interface Mockup

13.1 Landing Page

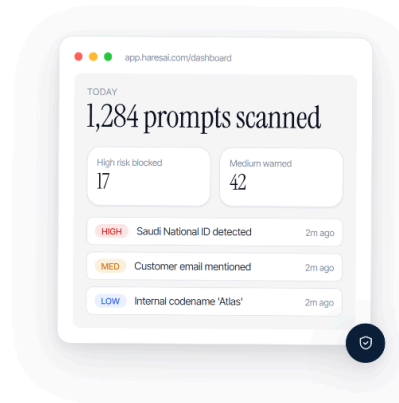
Enterprise DLP for generative AI

Protect your data *before* it leaves your hands.

Hares AI scans every prompt your team sends to ChatGPT, Gemini and Claude — flagging sensitive entities, blocking high-risk submissions, and offering safe rewrites in real time.

Get Started Free →

Learn More →



WHY TEAMS CHOOSE HARES AI

Three layers, one continuous defence.



Real-Time Detection

Pattern + AI hybrid scanning catches sensitive entities the moment they're typed — without slowing the flow.



Risk Classification

Every prompt is graded High / Medium / Low / None and routed to the right enforcement action automatically.



Audit Logs

Every scan, decision and override is recorded — exportable, filterable, and ready for compliance reviews.

HOW IT WORKS

From install to insight in under five minutes.

01



Install Extension

Deploy the Hares AI browser extension to every employee via your IdP or MDM.



02



Prompt is Scanned

Each outgoing prompt is parsed for sensitive entities — local-first, sub-100ms.



03



Risk is Flagged

High blocks, Medium warns, Low informs. Users get a clean rewrite in one click.

Stop leaks before they happen.

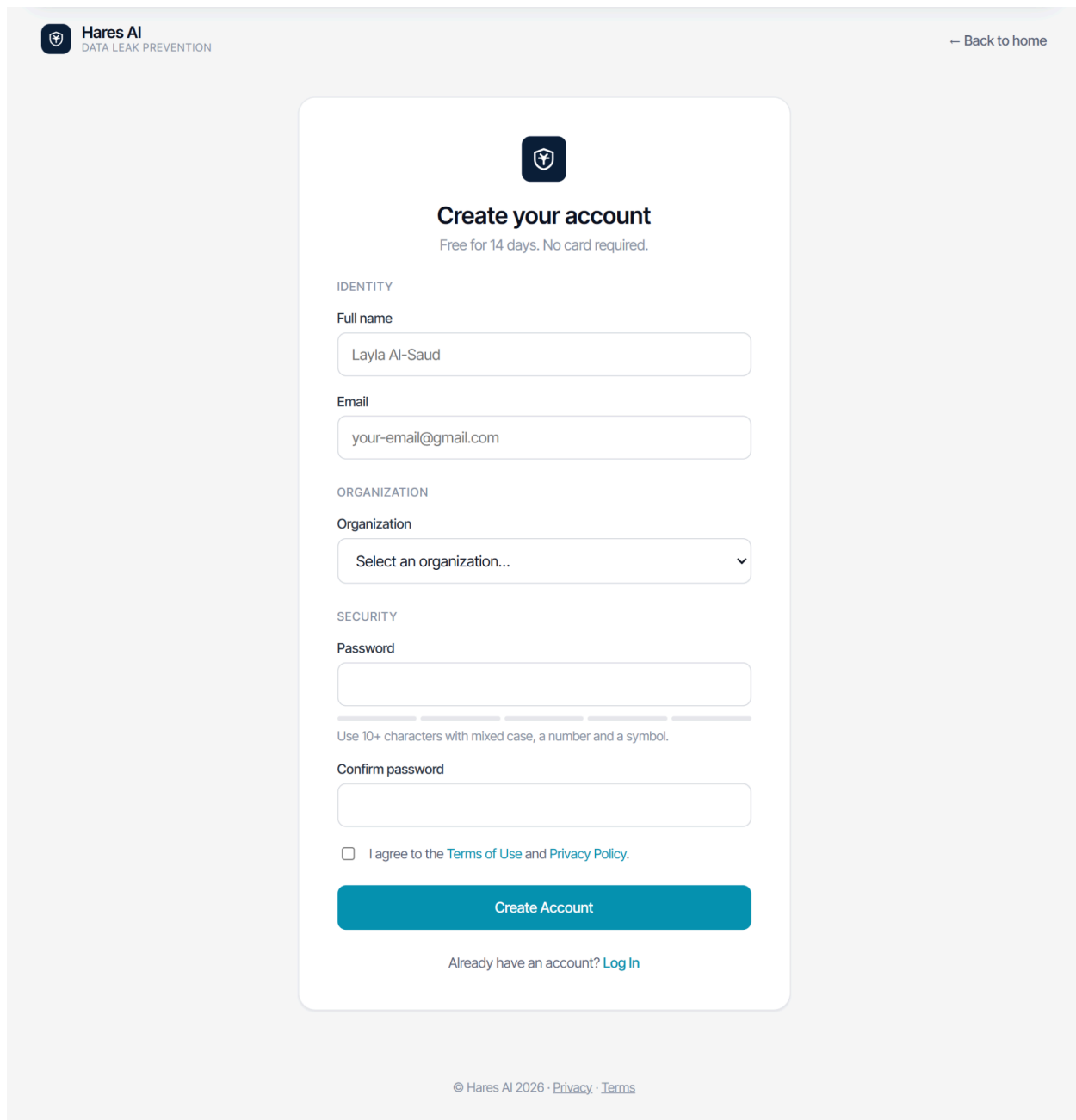
14-day free trial. No credit card required. Self-serve onboarding.

Get Started Free →

Book a demo

Figure 14. Landing Page.

13.2 Register Page



The screenshot shows the 'Create your account' page for Hares AI. The page has a light gray background. In the top left corner, there is a logo for 'Hares AI' with the tagline 'DATA LEAK PREVENTION'. In the top right corner, there is a link that says '← Back to home'. The main content is centered in a white rounded rectangle. At the top of this rectangle is a dark blue shield icon with a white keyhole. Below the icon, the text 'Create your account' is displayed in a bold, dark blue font, followed by the text 'Free for 14 days. No card required.' in a smaller, gray font. The form is divided into four sections: 'IDENTITY', 'ORGANIZATION', 'SECURITY', and a final section for terms and conditions. The 'IDENTITY' section has two input fields: 'Full name' (containing 'Layla Al-Saud') and 'Email' (containing 'your-email@gmail.com'). The 'ORGANIZATION' section has a dropdown menu for 'Organization' with the text 'Select an organization...' and a downward arrow. The 'SECURITY' section has two input fields: 'Password' and 'Confirm password'. Below the 'Confirm password' field is a checkbox with the text 'I agree to the Terms of Use and Privacy Policy.' At the bottom of the form is a large blue button labeled 'Create Account'. Below the button, there is a link that says 'Already have an account? Log In'. At the very bottom of the page, there is a footer that says '© Hares AI 2026 · Privacy · Terms'.

Hares AI
DATA LEAK PREVENTION

← Back to home

Create your account
Free for 14 days. No card required.

IDENTITY

Full name

Layla Al-Saud

Email

your-email@gmail.com

ORGANIZATION

Organization

Select an organization...

SECURITY

Password

Use 10+ characters with mixed case, a number and a symbol.

Confirm password

☐ I agree to the [Terms of Use](#) and [Privacy Policy](#).

Create Account

Already have an account? [Log In](#)

© Hares AI 2026 · [Privacy](#) · [Terms](#)

Figure 15. Register Page.

13.3 Login Page

**Hares AI**
DATA LEAK PREVENTION

← Back to home



Welcome back

Log in to your Hares AI account.

Email

Password

[Forgot password?](#)

Log In

OR

 Continue with Single Sign-On

Don't have an account? [Sign Up](#)

© Hares AI 2026 · [Privacy](#) · [Terms](#)

Figure 16. Login Page.

13.4 Forget Password Page

Hares AI
DATA LEAK PREVENTION

← Back to home



Reset your password

Enter the email address associated with your account.

Email

[Send Reset Link](#)

[← Back to Login](#)

© Hares AI 2026 · [Privacy](#) · [Terms](#)

Figure 17. Forget Password Page.

13.5 User Dashboard Page

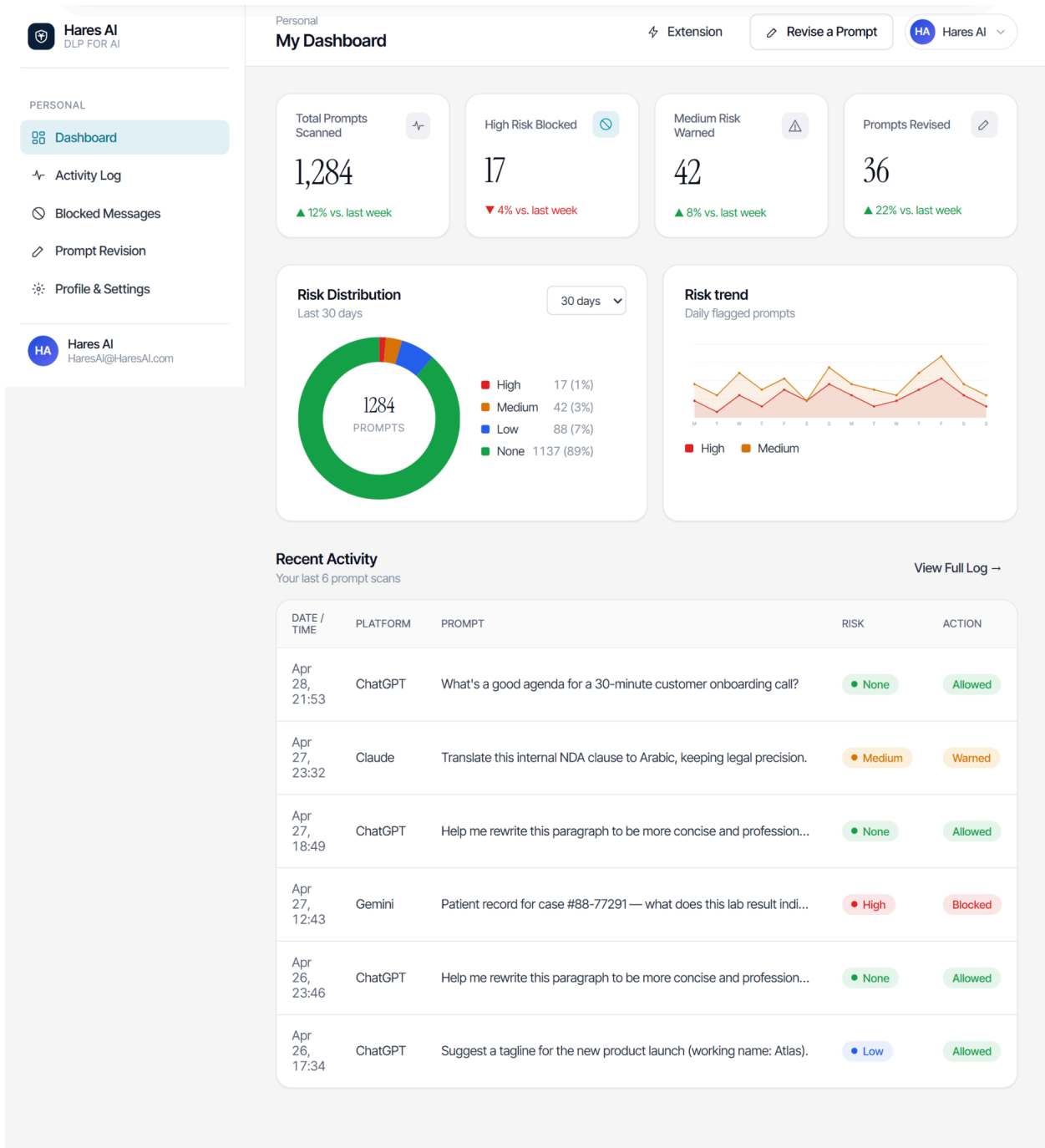


Figure 18. User Dashboard Page.

13.6 User Activity Log Page

 **Hares AI**
DLP FOR AI

PERSONAL

Dashboard

Activity Log

Blocked Messages

Prompt Revision

Profile & Settings

HA

Hares AI
HaresAI@HaresAI.com

Personal

My Activity Log

Export CSV

HA Hares AI

Last 30 days

All platforms

All risk levels

Search prompt text...

Clear

#	TIMESTAMP	PLATFORM	PROMPT PREVIEW	DETECTED ENTITIES	RISK	ACTION
01	Apr 28, 22:14	ChatGPT	What's a good agenda for a 30-minute customer onbo...	—	None	Allowed
02	Apr 27, 23:53	Claude	Translate this internal NDA clause to Arabic, keeping leg...	NDA clause	Medium	Warned
03	Apr 27, 19:10	ChatGPT	Help me rewrite this paragraph to be more concise and ...	—	None	Allowed
04	Apr 27, 13:04	Gemini	Patient record for case #88-77291 — what does this lab...	Patient record #88-77291	High	Blocked
05	Apr 27, 00:07	ChatGPT	Help me rewrite this paragraph to be more concise and ...	—	None	Allowed
06	Apr 26, 17:55	ChatGPT	Suggest a tagline for the new product launch (working ...	Atlas	Low	Allowed
07	Apr 26, 16:02	Gemini	My credit card 4532 1488 0343 6467 was declined — ...	4532 1488 03...	High	Blocked
08	Apr 26, 07:10	ChatGPT	Summarize this candidate's CV: Faisal Younis, faisal.y@...	Faisal Younis faisal.y@ema... +1	Medium	Warned

Showing 1-8 of 80

— Prev 1 2 3 Next —

Figure 19. User Activity Log Page.

13.7 User Blocked Prompt Page

The screenshot shows the 'Blocked Messages' page for a user named 'Hares AI'. The page displays a list of blocked prompts with columns for Timestamp, Platform, Prompt, Blocked Entities, and Reason. A summary at the top indicates that 5 prompts have been blocked. The left sidebar contains navigation links for Dashboard, Activity Log, Blocked Messages (selected), Prompt Revision, and Profile & Settings. The top right corner has an 'Export CSV' button and a user profile dropdown.

TIMESTAMP	PLATFORM	PROMPT	BLOCKED ENTITIES	REASON	
Apr 27, 13:11	Gemini	Patient record for case #88-77291 — what does this lab result L...	Health record Case ID	Sensitive PII / financial data detected	Revise
Apr 26, 16:09	Claude	My credit card #532 1488 0343 6467 was declined — can you ...	Credit Card Number	Sensitive PII / financial data detected	Revise
Apr 25, 09:53	ChatGPT	Draft a refund email to customer Mariam Al-Hashimi (ID 110293...	Person Name Saudi National ID IBAN	Sensitive PII / financial data detected	Revise
Apr 25, 05:24	Claude	Draft a refund email to customer Mariam Al-Hashimi (ID 110293...	Person Name Saudi National ID IBAN	Sensitive PII / financial data detected	Revise
Apr 24, 23:20	Gemini	Patient record for case #88-77291 — what does this lab result L...	Health record Case ID	Sensitive PII / financial data detected	Revise

Figure 20. User Blocked Prompt Page.

13.8 User Revision Prompt Page

The screenshot shows the 'Prompt Revision' page for a user named 'Hares AI'. The page displays a comparison between the 'Original Prompt' (marked as 'High Risk') and the 'Revised Prompt' (marked as 'Safe'). The original prompt contains sensitive information like a customer's name and ID. The revised prompt is a safer version that uses placeholders. Below the prompts, there are buttons for 'Use This Version', 'Copy Revised', 'Edit Manually', and 'Submit to ChatGPT'. A 'Suggested Structured Version' is also provided, detailing the goal, tone, required fields, and constraints for the refund email.

PERSONAL

- Dashboard
- Activity Log
- Blocked Messages
- Prompt Revision**
- Profile & Settings

Hares AI
HaresAI@HaresAI.com

Personal

Prompt Revision

✓ 3 entities removed

HA Hares AI

Original Prompt High Risk

Draft a refund email to customer Mariam Al-Hashimi (ID 1102936453 , IBAN SA03 8000 0000 6080 1016 7519) for invoice 4421.

FLAGGED ENTITIES

- Person Name: Mariam Al-Hashimi
- Saudi National ID: 1102936453
- IBAN: SA03 8000 0000 6...

Revised Prompt - editable Safe

Draft a refund email to a customer (referred to as [Customer Name]) regarding their invoice. Reference the invoice number and apologise for the delay; confirm the refund will be processed within 5 business days.

Use This Version

Copy Revised Edit Manually

Submit to ChatGPT

Suggested Structured Version Copy

Goal: Customer refund email
Tone: Polite, apologetic
Required fields:
• [Customer Name]
• [Invoice Number]
• [Refund Window: 5 business days]
Constraints: Do not include national IDs, IBANs, or any financial identifiers.

Figure 21. User Revision Prompt Page.

13.9 User Profile Page

1 Landing2 Login3 Register4 Forgot5 Dashboard6 Activity Log7 Blocked8 Revision9 Profile10 Dept Dashboard11 Dept Log

PERSONAL

Dashboard

Activity Log

Blocked Messages

Prompt Revision

Profile & Settings

HA

Hares AI

HaresAI@HaresAI.com

Personal Information

How you appear inside Hares AI.

Full name

Hares AI

Email

HaresAI@HaresAI.com

Organization

Acme Corp

Department

Engineering

Save changes

Change Password

Use a strong, unique password.

Current password

New password

Confirm new password

Update password

Notification Preferences

Email alerts for High Risk blocks

Sent every time a high-risk prompt is blocked.

Weekly activity summary

Receive a Monday-morning email with last week's stats.

Extension Settings

Synced with your browser extension.

Show Low Risk info banners

Display the small bottom-right toast for Low risk detections.

Danger Zone

Irreversible actions.

Delete my account

Permanently remove your account and all associated audit logs.

Delete account

Figure 22. User Profile Page.

13.10 Department Admin Dashboard Page

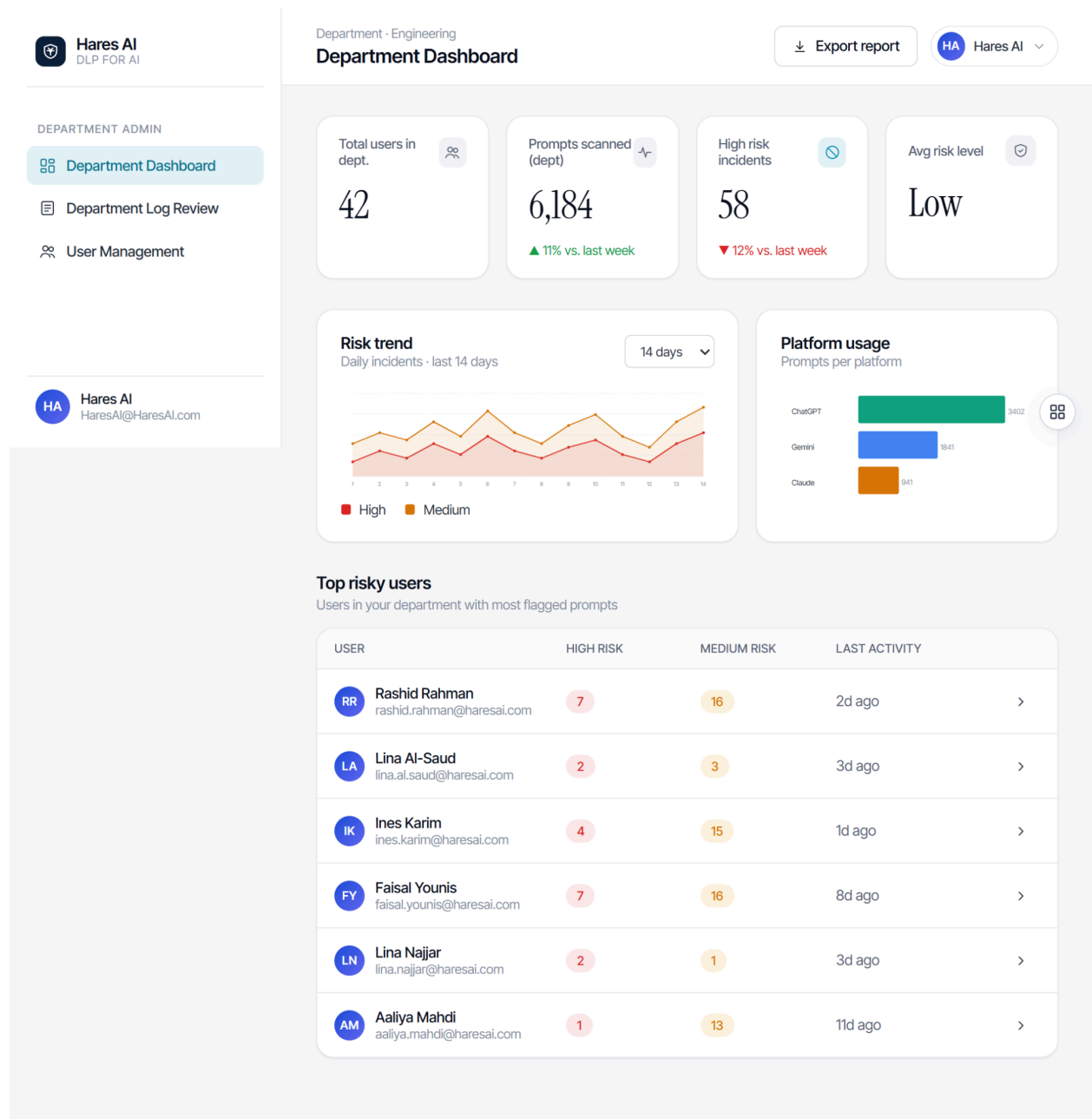


Figure 23. Department Admin Dashboard Page.

13.11 Department Admin Log Page

profile

10 Dept Dashboard

11 Dept Log

12 Dept Users

13 Org Dashboard

14 Full Log

15 All Users

16 Roles

17 Departments

18 Configuration

DEPARTMENT ADMIN

Department Dashboard

Department Log Review

User Management

HA

Hares AI

HaresAI@HaresAI.com

Last 30 days

All platforms

All risk levels

Clear

Search prompt text...

#	TIMESTAMP	PLATFORM	DETECTED ENTITIES	RISK	ACTION
01	Apr 28, 22:35	ChatGPT	—	None	Allowed
02	Apr 28, 00:14	Claude	NDA clause	Medium	Warned
03	Apr 27, 19:31	ChatGPT	—	None	Allowed
04	Apr 27, 13:25	Gemini	Patient record #88-77291	High	Blocked
05	Apr 27, 00:28	ChatGPT	—	None	Allowed
06	Apr 26, 18:16	ChatGPT	Atlas	Low	Allowed
07	Apr 26, 16:23	Gemini	4532 1488 03...	High	Blocked
08	Apr 26, 07:31	ChatGPT	Faisal Younis faisal.y@ema... +1	Medium	Warned

Showing 1-8 of 80

← Prev

1

2

3

Next →

Figure 24. Department Admin Log Page.

13.12 Department User Management Page

**Hares AI**
DLP FOR AI

DEPARTMENT ADMIN

 Department Dashboard

 Department Log Review

 **User Management**

**Hares AI**
HaresAI@HaresAI.com

Department · Engineering

User Management — Engineering

5 Active · 0 Inactive

 Hares AI

Q Search by name or email...

 All statuses

NAME	EMAIL	STATUS	LAST ACTIVE	ACTIONS
 Fatimah Habib	fatimah.habib@haresai.com	Active	12d ago	Deactivate
 Maya Halabi	maya.halabi@haresai.com	Active	9d ago	Deactivate
 Fatimah Halabi	fatimah.halabi@haresai.com	Active	8d ago	Deactivate
 Reem Dahan	reem.dahan@haresai.com	Active	11d ago	Deactivate
 Aaliya Khouri	aaliya.khouri@haresai.com	Active	13d ago	Deactivate

Figure 25. Department User Management Page.

13.13 System Admin Dashboard Page

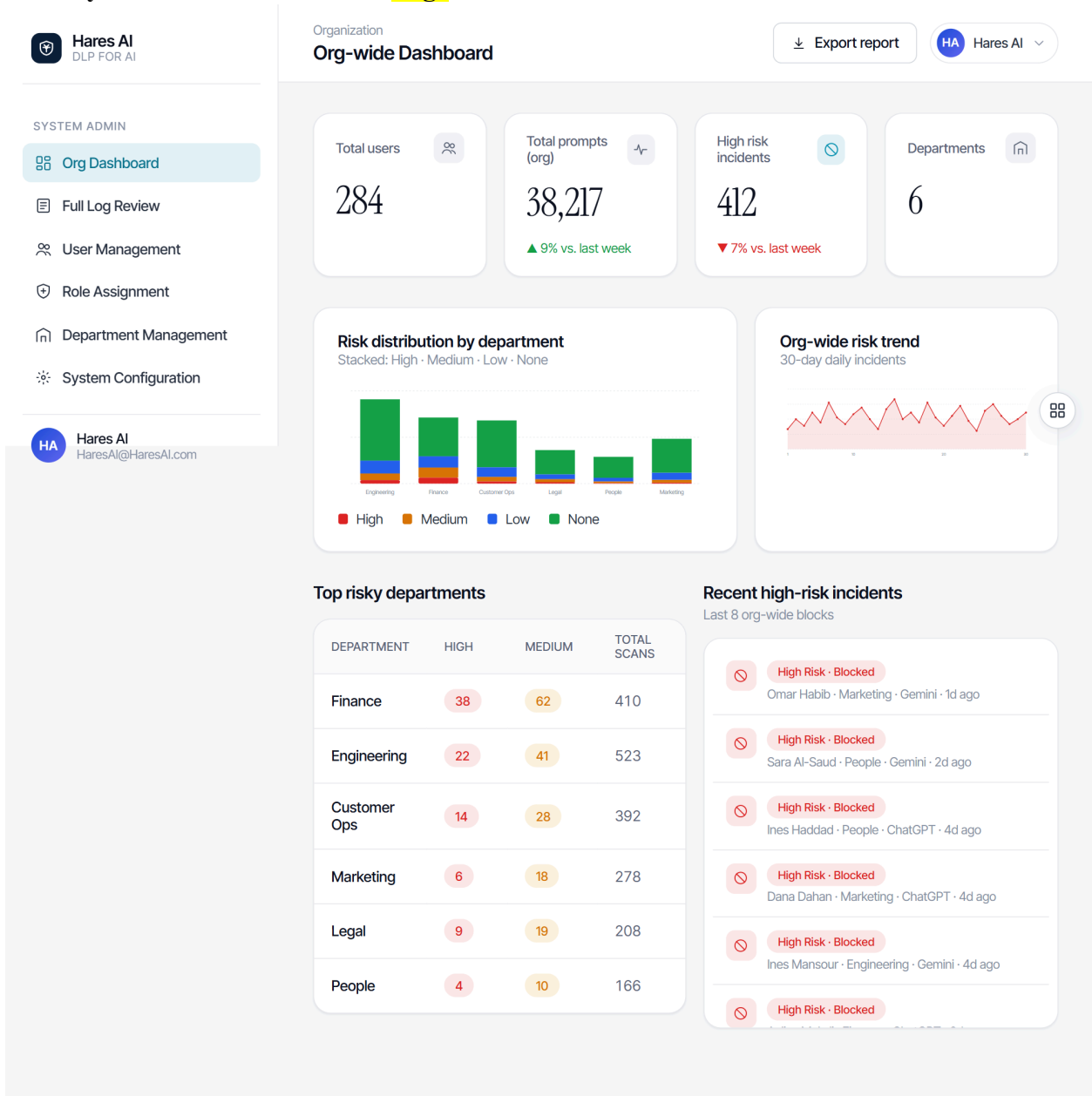


Figure 26. System Admin Dashboard Page.

13.14 System Admin Full Log Page


Hares AI
B.L.P. FOR AI

SYSTEM ADMIN
 Org Dashboard
Full Log Review
 User Management
 Role Assignment
 Department Management
 System Configuration


Hares AI
HaresAI@HaresAI.com

Organization
Full Activity Log

Export CSV
 
 Hares AI

All departments
 All users
 Last 30 days
 All platforms
 All risk
 Search prompt text...

☐	TIMESTAMP	DEPARTMENT	USER	PLATFORM	RISK	ACTION
☐	Apr 28, 22:48	Marketing	 Lina Al-Saud	ChatGPT	None	Allowed
☐	Apr 28, 00:27	Engineering	 Rashid Dahan	Claude	Medium	Warned
☐	Apr 27, 19:44	People	 Rashid Mansour	ChatGPT	None	Allowed
☐	Apr 27, 13:38	Marketing	 Omar Habib	Gemini	High	Blocked
☐	Apr 27, 00:41	Customer Ops	 Sara Habib	ChatGPT	None	Allowed
☐	Apr 26, 18:29	Legal	 Maya Halabi	ChatGPT	Low	Allowed
☐	Apr 26, 16:36	People	 Sara Al-Saud	Gemini	High	Blocked
☐	Apr 26, 07:44	Legal	 Omar Aziz	ChatGPT	Medium	Warned
☐	Apr 25, 20:46	People	 Omar Aziz	Gemini	Medium	Warned
☐	Apr 25, 10:20	People	 Ines Haddad	ChatGPT	High	Blocked

Figure 27. System Admin Full Log Page.

13.15 System Admin User Management Page

 Hares AI
DLP FOR AI

SYSTEM ADMIN

Org Dashboard

Full Log Review

User Management

Role Assignment

Department Management

System Configuration

HA

Hares AI
HaresAI@HaresAI.com

Organization

User Management

+ Add User

HA Hares AI

All departments

All roles

All statuses

Search by name or email...

NAME	EMAIL	DEPARTMENT	ROLE	STATUS	LAST ACTIVE		
RR Rashid Rahman	rashid.rahman@haresai.com	Customer Ops	System Admin	Active	2d ago	Edit	...
LA Lina Al-Saud	lina.al.saud@haresai.com	Finance	Department Admin	Active	3d ago	Edit	...
IK Ines Karim	ines.karim@haresai.com	Marketing	Department Admin	Active	1d ago	Edit	...
FY Faisal Younis	faisal.younis@haresai.com	Legal	Department Admin	Active	8d ago	Edit	...
LN Lina Najjar	lina.najjar@haresai.com	People	User	Active	3d ago	Edit	...
AM Aaliya Mahdi	aaliya.mahdi@haresai.com	Legal	User	Active	11d ago	Edit	...
FS Faisal Saleh	faisal.saleh@haresai.com	Finance	User	Active	4d ago	Edit	...
OA Omar Al-Saud	omar.al.saud@haresai.com	Marketing	User	Inactive	1d ago	Edit	...
HM Hamza Mansour	hamza.mansour@haresai.com	Marketing	User	Active	2d ago	Edit	...
FH Fatimah Habib	fatimah.habib@haresai.com	Engineering	User	Active	12d ago	Edit	...
MH Maya Halabi	maya.halabi@haresai.com	Engineering	User	Active	9d ago	Edit	...
FH Fatimah Halabi	fatimah.halabi@haresai.com	Engineering	User	Active	8d ago	Edit	...
RD Reem Dahan	reem.dahan@haresai.com	Engineering	User	Active	11d ago	Edit	...
AK Aaliya Khouri	aaliya.khouri@haresai.com	Engineering	User	Active	13d ago	Edit	...
DR Dana Rahman	dana.rahman@haresai.com	Finance	User	Inactive	4h ago	Edit	...
DY Dana Younis	dana.younis@haresai.com	People	User	Active	2d ago	Edit	...

Figure 28. System Admin User Management Page.

13.16 System admin Role Assignment Page

Hares AI

DLP FOR AI

SYSTEM ADMIN

Org Dashboard

Full Log Review

User Management

Role Assignment

Department Management

System Configuration

HA

Hares AI

HaresAI@HaresAI.com

Organization

Role Assignment

0 pending changes

Apply all changes

HA Hares AI

Q Search users...

USER	CURRENT	NEW ROLE
RR Rashid Rahman rashid.rahman@haresai.com	System Admin	System Admin
LA Lina Al-Saud lina.al.saud@haresai.com	Department Admin	Department Admin
IK Ines Karim ines.karim@haresai.com	Department Admin	Department Admin
FY Faisal Younis faisal.younis@haresai.com	Department Admin	Department Admin
LN Lina Najjar lina.najjar@haresai.com	User	User
AM Aaliya Mahdi aaliya.mahdi@haresai.com	User	User
FS Faisal Saleh faisal.saleh@haresai.com	User	User
QA Omar Al-Saud omar.al.saud@haresai.com	User	User
HM Hamza Mansour hamza.mansour@haresai.com	User	User
FH Fatimah Habib fatimah.habib@haresai.com	User	User
MH Maya Halabi maya.halabi@haresai.com	User	User
FH Fatimah Halabi fatimah.halabi@haresai.com	User	User
RD Reem Dahan reem.dahan@haresai.com	User	User
AK Aaliya Khouri aaliya.khouri@haresai.com	User	User

Recent role changes

Last 30 days

Reem Karim

User

Department Admin

by Hares AI · 2h ago

Bilal Halabi

Department Admin

User

by Hares AI · Yesterday

Yusuf Najjar

User

Department Admin

by Hares AI · 3d ago

Faisal Younis

User

User

by Hares AI · 1w ago

Figure 29. System Admin Role Assignment Page.

13.17 System Admin Departments Management Page



Hares AI

DLP FOR AI

SYSTEM ADMIN

 Org Dashboard

 Full Log Review

 User Management

 Role Assignment

 Department Management

 System Configuration

 Hares AI

Hares AI

HaresAI@HaresAI.com

Organization

Departments

+ Add Department

 Hares AI

DEPARTMENT	MEMBERS	ADMIN	CREATED		
 Engineering	42 users	 Hares AI	Mar 2024	Edit	
 Finance	28 users	 Khalid Mansour	Mar 2024	Edit	
 Customer Ops	36 users	 Reem Karim	Apr 2024	Edit	
 Legal	14 users	 Hadi Sabbagh	May 2024	Edit	
 People	18 users	 Maya Rahman	Jul 2024	Edit	
 Marketing	22 users	 Sara Othman	Sep 2024	Edit	



Figure 30. System Departments Management Page.

13.18 High Risk POP-UP Message

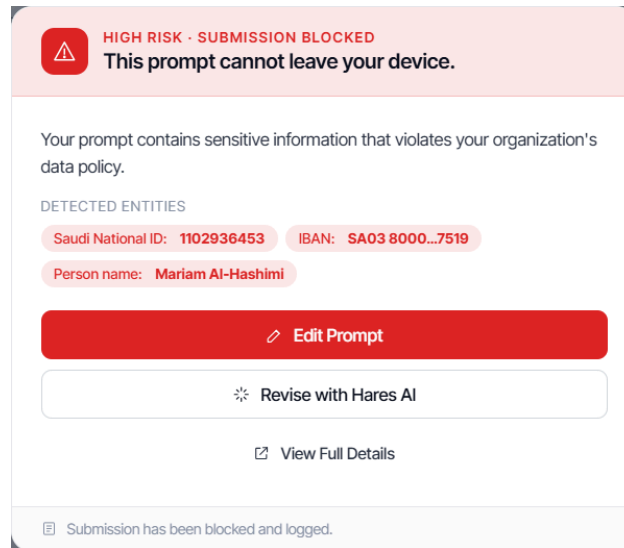


Figure 31. High Risk POP-UP Message.

13.19 Medium Risk POP-UP Message

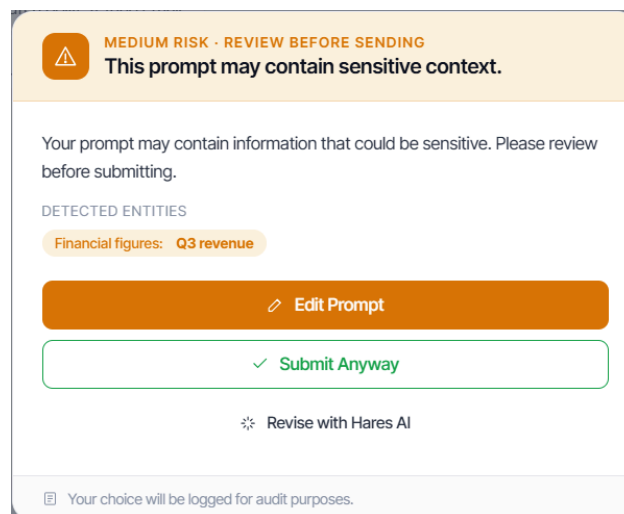


Figure 32. Medium Risk POP-UP Message .

13.20 Low Risk POP-UP Message

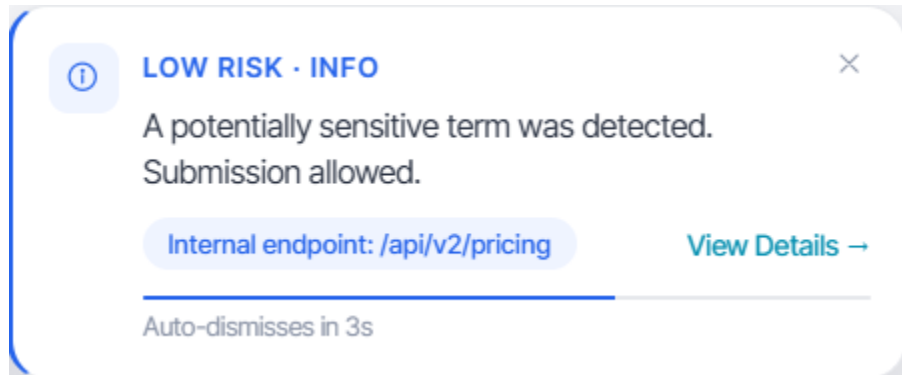


Figure 33. Low Risk POP-UP Message.

13.21 Revision Prompt UI

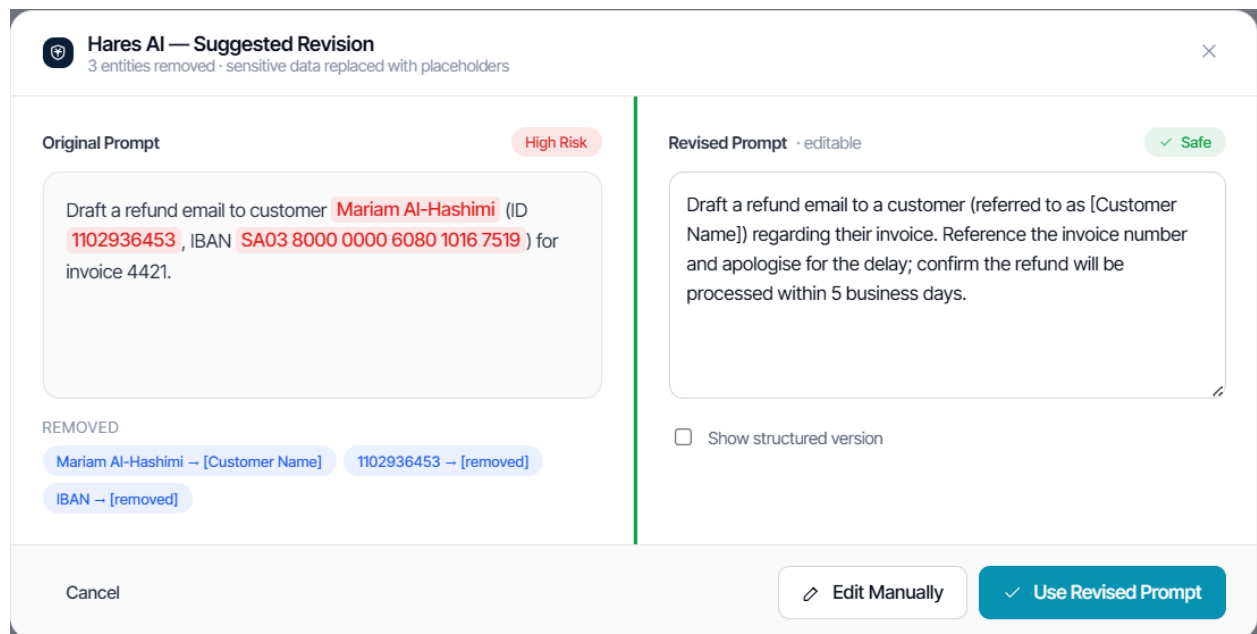


Figure 34. Revision Prompt UI.

14. Database Schema

14.1 ER diagram

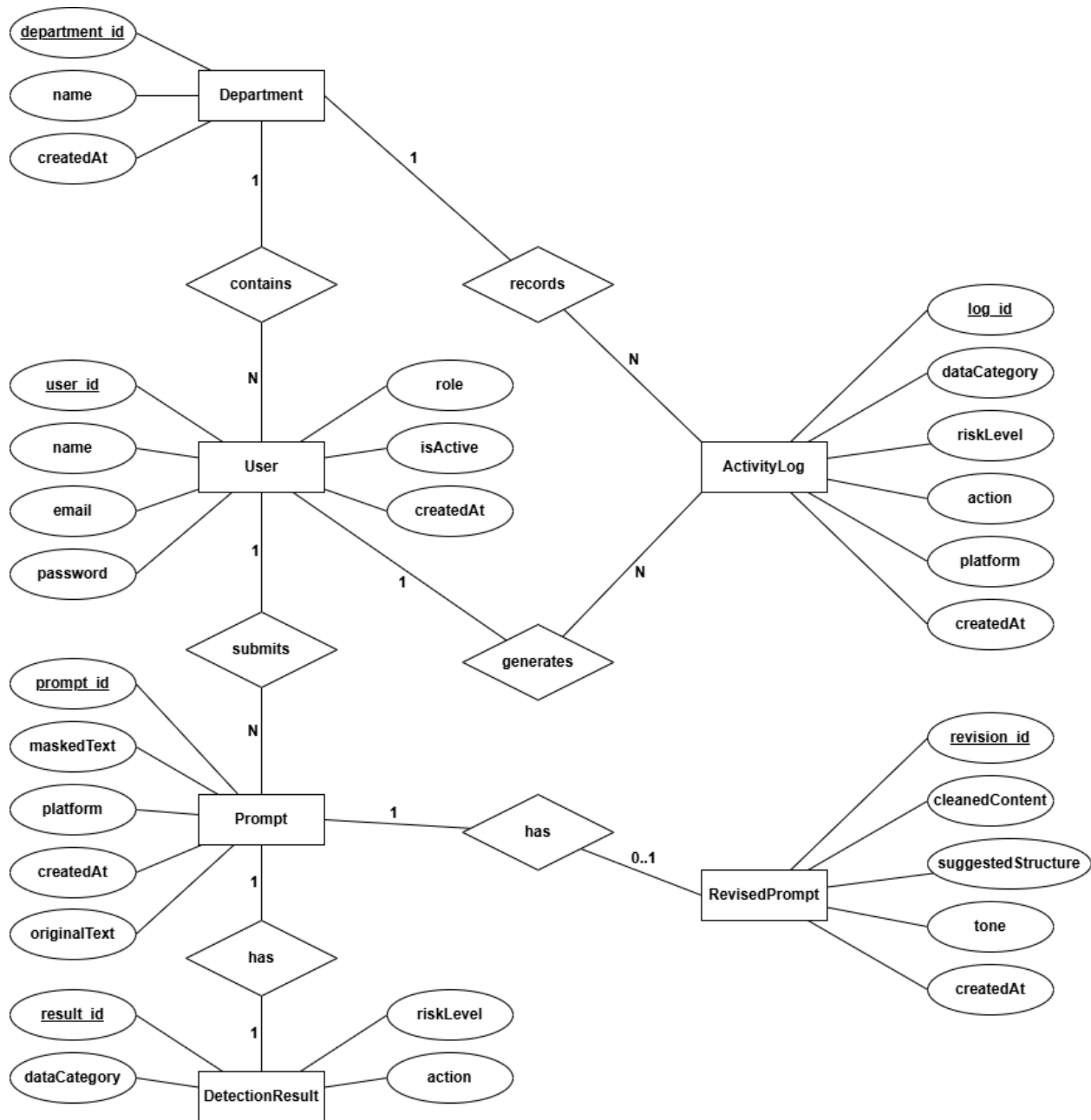


Figure 35. ER Diagram.

14.2 Database schema

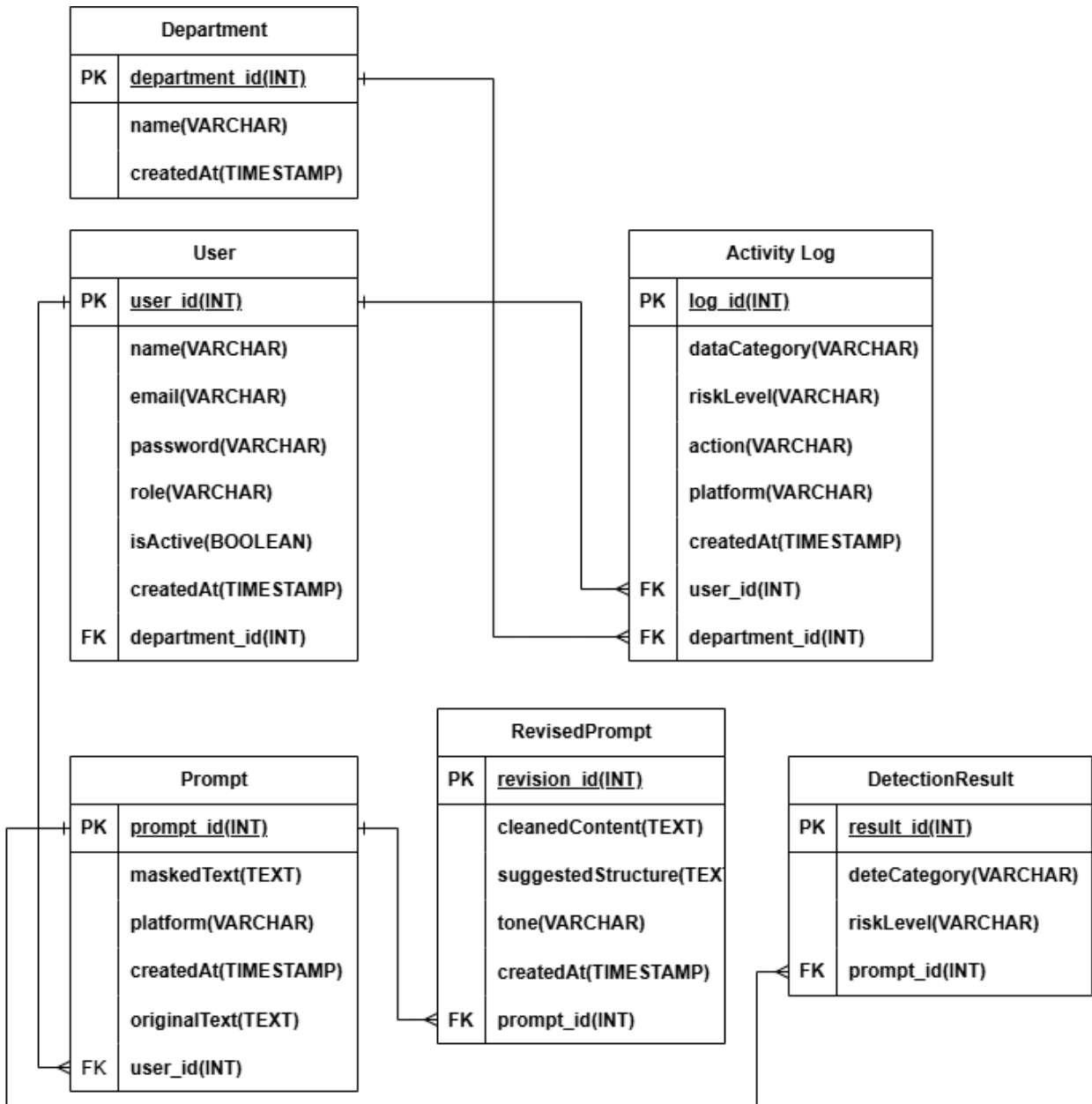


Figure 36. Database Schema.

15. Algorithms

15.1 Local Regex-Based Sensitive Data Detection

```
FUNCTION LocalRegexDetection(rawPrompt : String) : List<Detection>

  // Step 1: Initialise empty results container
  detections ← []

  // Step 2: Load pattern dictionary from Chrome Storage
  //   Patterns are stored externally to allow updates without redeployment
  patternDictionary ← GetRegexPatterns() // Map<Category, CompiledRegex>

  // Step 3: Apply every pattern to the raw prompt
  FOR EACH (category, pattern) IN patternDictionary DO
    matches ← pattern.FindAllMatches(rawPrompt) // non-overlapping

    FOR EACH match IN matches DO
      detection ← {
        category : category,    // e.g. API_KEY, EMAIL, CREDIT_CARD
        value    : match.Value,
        startIndex : match.StartIndex,
        endIndex  : match.EndIndex
      }
      ADD detection TO detections
    END FOR
  END FOR

  // Step 4: Remove duplicate / fully-overlapping detections
  //   If detection B is fully contained within detection A, keep A and discard B
  detections ← RemoveOverlaps(detections)

  // Step 5: Sort ascending by startIndex for downstream masking (Algorithm 15.2)
  SORT detections BY startIndex ASCENDING

  RETURN detections

END FUNCTION

// Helper

FUNCTION RemoveOverlaps(detections : List<Detection>) : List<Detection>
  result ← []
  FOR EACH d IN detections DO
    dominated ← FALSE
```

```

FOR EACH other IN detections DO
  IF other ≠ d AND other.startIndex ≤ d.startIndex
    AND other.endIndex ≥ d.endIndex THEN
    dominated ← TRUE
    BREAK
  END IF
END FOR
IF NOT dominated THEN ADD d TO result END IF
END FOR
RETURN result
END FUNCTION

```

15.2 Sensitive Data Masking

```

FUNCTION MaskSensitiveData(rawPrompt : String,
  detections : List<Detection>) : String

  // Step 1: Work on a mutable copy
  maskedPrompt ← rawPrompt

  // Step 2: Iterate in DESCENDING startIndex order
  //   Replacing from the end backwards prevents index positions
  //   of earlier detections from shifting after each replacement
  FOR EACH detection IN REVERSE(detections) DO
    placeholder ← '[' + detection.category + ']'
    maskedPrompt ← ReplaceSubstring(
      maskedPrompt,
      detection.startIndex,
      detection.endIndex,
      placeholder
    )
  END FOR

  // Step 3: Return the fully masked text
  //   At this point NO raw sensitive value remains in the string
  RETURN maskedPrompt

END FUNCTION

```

15.3 Risk Classification and Policy Decision

```

FUNCTION ClassifyRiskAndDecide(detectedCategories : List<String>,
  maskedPrompt : String,
  activePolicy : PolicyConfig)
  : ClassificationResult

  // Step 1: Load the fixed category → risk level mapping
  //   HIGH : Authentication & Secrets, Financial & Banking,
  //   Personal Identity, Cryptography & Certificates,

```

```

//      Developer Secrets
//  MEDIUM : Contact Info, Network & System Data, Location,
//      Legal & Government, Vehicle, Corporate & Document,
//      File & Path, Digital Fingerprints, Shipping & Logistics,
//      Dates & Time, Logs & Headers, Numeric Sensitive,
//      Education-Related
//  LOW   : Keyword-Based Sensitive Patterns only
riskMapping ← GetCategoryRiskMapping() // Map<Category, RiskLevel>

// Step 2: Find the highest risk level across all detected categories
highestRisk ← NONE
FOR EACH cat IN detectedCategories DO
    categoryRisk ← riskMapping.GetRiskLevel(cat)
    IF categoryRisk > highestRisk THEN
        highestRisk ← categoryRisk
    END IF
END FOR

// Step 3: Determine whether Gemini secondary classification is needed
//  Trigger condition: NO categories were detected locally AND
//      the masked prompt exceeds 200 characters
requiresGemini ← (highestRisk = NONE) AND (LENGTH(maskedPrompt) > 200)

// Step 4: Apply the policy to translate risk level → preliminary action
//  Default mapping : HIGH → BLOCK | MEDIUM → WARN | LOW → INFO | NONE → ALLOW
//  Policy overrides : e.g. force WARN instead of BLOCK for a specific department
preliminaryAction ← activePolicy.GetActionForRiskLevel(highestRisk)

RETURN {
    riskLevel    : highestRisk,
    action       : preliminaryAction,
    categories    : detectedCategories, // full list preserved for logging
    requiresGemini : requiresGemini
}

END FUNCTION

```

15.4 Secondary Classification via Gemini API

```

FUNCTION GeminiSecondaryClassification(maskedPrompt : String,
                                     localResult : ClassificationResult,
                                     activePolicy : PolicyConfig)
    : FinalClassificationResult

// Step 1: Build a single deterministic classification prompt
//  Asking for JSON output in one call avoids a second round-trip
geminiPrompt ← BuildClassificationPrompt(maskedPrompt)
//  The prompt instructs the model to respond ONLY with valid JSON:
//  { "riskLevel": "HIGH|MEDIUM|LOW|NONE", "categories": ["..."] }

```

```

// No preamble, no markdown, no explanation.

// Step 2: Configure API call with tight generation parameters
requestBody ← {
  contents : [{ parts: [{ text: geminiPrompt }] }],
  generationConfig : {
    temperature : 0.1,
    topK : 1,
    topP : 0.9,
    maxOutputTokens : 60 // sufficient for JSON risk + categories
  }
}

// Step 3: Call Gemini API with 400 ms timeout
TRY
  response ← CallGeminiAPI(requestBody, timeoutMs = 800)
CATCH TimeoutException OR NetworkException
  // Fallback: return the local result unchanged
  RETURN {
    riskLevel : localResult.riskLevel,
    action : localResult.action,
    categories : localResult.categories,
    source : 'LOCAL_ONLY_FALLBACK'
  }
END TRY

// Step 4: Parse the structured JSON response safely
TRY
  aiResult ← ParseJSON(response) // { riskLevel, categories }
  IF aiResult IS INVALID OR aiResult.riskLevel IS NOT IN [HIGH,MEDIUM,LOW,NONE] THEN
    THROW ParseException
  END IF
CATCH ParseException
  // Malformed response — fall back to local result
  RETURN {
    riskLevel : localResult.riskLevel,
    action : localResult.action,
    categories : localResult.categories,
    source : 'LOCAL_ONLY_FALLBACK'
  }
END TRY

// Step 5: Merge local and AI results; higher risk level takes precedence
finalRisk ← MAX(localResult.riskLevel, aiResult.riskLevel)
finalCategories ← UNION(localResult.categories, aiResult.categories)
finalAction ← activePolicy.GetActionForRiskLevel(finalRisk)

RETURN {
  riskLevel : finalRisk,
  action : finalAction,
  categories : finalCategories,

```

```

    source    : 'LOCAL+GEMINI'
  }

END FUNCTION

```

15.5 Policy Enforcement Decision

```

FUNCTION EnforcePolicyDecision(finalResult : FinalClassificationResult,
                               activePolicy : PolicyConfig)
  : EnforcementDecision

  // Step 1: Apply department-level overrides on top of the base policy
  // Example overrides stored in PolicyConfig:
  // - 'Finance' department: treat MEDIUM as BLOCK
  // - 'Marketing' department: treat HIGH as WARN (acknowledge-only)
  action ← activePolicy.ApplyOverrides(finalResult.action,
                                       finalResult.categories)

  // Step 2: Map action to the correct UI command
  SWITCH action
  CASE BLOCK:
    uiInstruction ← {
      type      : 'BLOCK_SUBMISSION',
      displayMode : 'HIGH_RISK_POPUP',
      message    : 'Submission blocked. Remove sensitive data to continue.',
      showRevise : TRUE
    }

  CASE WARN:
    uiInstruction ← {
      type      : 'WARN_USER',
      displayMode : 'MEDIUM_RISK_POPUP',
      message    : 'Sensitive context detected. Confirm to proceed.',
      requiresAck : TRUE,
      showRevise : TRUE
    }

  CASE ALLOW WITH INFO:
    uiInstruction ← {
      type      : 'INFO_TOAST',
      displayMode : 'LOW_RISK_TOAST',
      message    : 'Low-risk term detected. Submission allowed.',
      autoDismiss : 3000 // milliseconds
    }

  CASE ALLOW:
    uiInstruction ← {
      type      : 'ALLOW_SUBMISSION',
      displayMode : 'NONE'
    }

```

```

    }
END SWITCH

// Step 3: Return the complete enforcement decision
RETURN {
    action      : action,
    riskLevel   : finalResult.riskLevel,
    categories  : finalResult.categories,
    uiInstruction : uiInstruction
}

END FUNCTION

```

15.6 Compliant Revision Generation

```

FUNCTION GenerateCompliantRevision(maskedPrompt : String,
    activePolicy : PolicyConfig,
    tone        : String = 'professional')
    : RevisedPrompt

// Step 1: Build a single combined instruction for one Gemini API call
//   Requesting both versions in one call avoids a second round-trip
//   and keeps total latency within the 1-second UX budget
combinedInstruction ← BuildRevisionPrompt(
    maskedPrompt,
    tone,
    instructions := [
        'cleanVersion  : Rewrite the prompt removing all [BRACKETED_TAGS].',
        '               Preserve the original intent without adding new sensitive data.',
        'structuredVersion: Restructure cleanVersion with these labelled sections:',
        '               ROLE | CONTEXT | TASK | OUTPUT FORMAT | TONE.'
    ]
)
// The instruction also specifies: respond ONLY with valid JSON:
// { "cleanVersion": "...", "structuredVersion": "..." }

// Step 2: Call Gemini API with a single request
requestBody ← {
    contents : [{ parts: [{ text: combinedInstruction }] }],
    generationConfig : {
        temperature : 0.4, // slight creativity for natural prose
        maxOutputTokens : 800
    }
}

TRY
    response ← CallGeminiAPI(requestBody, timeoutMs = 600)
CATCH TimeoutException OR NetworkException
    // Graceful degradation: signal that revision is unavailable

```

```

    RETURN {
        cleanVersion    : NULL,
        structuredVersion : NULL,
        tone            : tone,
        available       : FALSE,
        reason          : 'GEMINI_UNAVAILABLE'
    }
END TRY

// Step 3: Parse the combined JSON response
TRY
    revisionData ← ParseJSON(response) // { cleanVersion, structuredVersion }
    IF revisionData.cleanVersion IS EMPTY OR NULL THEN
        THROW ParseException
    END IF
CATCH ParseException
    RETURN {
        cleanVersion    : NULL,
        structuredVersion : NULL,
        tone            : tone,
        available       : FALSE,
        reason          : 'INVALID_RESPONSE'
    }
END TRY

// Step 4: Return both revision versions
RETURN {
    cleanVersion    : revisionData.cleanVersion,
    structuredVersion : revisionData.structuredVersion,
    tone            : tone,
    available       : TRUE
}
}

END FUNCTION

```

15.7 Anonymized Activity Logging

```

FUNCTION CreateAnonymizedLog(userId      : String,
                             departmentId : String,
                             platform     : Platform,
                             riskLevel    : RiskLevel,
                             action       : ActionType,
                             categories    : List<String>,
                             classificationSource : String)
    : ActivityLog

// Step 1: Construct the log entry with only non-sensitive metadata
// ABSOLUTELY NO raw text, masked text, or placeholder values
logEntry ← {
    logId      : GenerateUUID(),
    userId     : userId,
    departmentId : departmentId, // required for dept-scoped dashboard

```

```

timestamp      : GetCurrentUTCTimestamp(),
platform       : platform,      // CHATGPT | GEMINI | CLAUDE
riskLevel      : riskLevel,     // HIGH | MEDIUM | LOW | NONE
action         : action,        // BLOCKED | WARNED | ALLOWED
categories      : categories,    // category names only — no values
classificationSource : classificationSource // LOCAL | LOCAL+GEMINI | LOCAL_ONLY_FALLBACK
}

// Step 2: Persist to Firebase Firestore via the Data Access Layer
TRY
  PersistLog(logEntry)
CATCH PersistenceException AS e
  // Log the internal error but do NOT surface it to the user or block interception
  LogInternalError('PERSISTENCE_FAILURE', e.message)
END TRY

// Step 3: Trigger a non-blocking, async dashboard statistics update
// This must NOT block the main interception response path
ASYNC TriggerDashboardCacheUpdate(departmentId, riskLevel, action, platform)

// Step 4: Return the log entry (used for optional immediate UI update)
RETURN logEntry

END FUNCTION

```

15.8 End-to-End Pipeline

```

FUNCTION InterceptAndProcessPrompt(rawPrompt : String,
    userId : String,
    departmentId : String,
    platform : Platform,
    activePolicy : PolicyConfig,
    userTone : String = 'professional')

// — Phase 1: Local Detection & Masking —————
detections ← LocalRegexDetection(rawPrompt) // 15.1
maskedPrompt ← MaskSensitiveData(rawPrompt, detections) // 15.2
categories ← EXTRACT_UNIQUE_CATEGORIES(detections)

// — Phase 2: Classification —————
localResult ← ClassifyRiskAndDecide(categories,
    maskedPrompt,
    activePolicy) // 15.3

IF localResult.requiresGemini THEN

```

```

    finalResult ← GeminiSecondaryClassification(maskedPrompt,
                                              localResult,
                                              activePolicy) // 15.4
ELSE
    finalResult ← localResult // skip Gemini; result is definitive
END IF

// — Phase 3: Enforcement —————
decision ← EnforcePolicyDecision(finalResult, activePolicy) // 15.5
SEND_TO_CONTENT_SCRIPT(decision.uiInstruction)

// — Phase 4: Revision (parallel, only when needed) —————
IF decision.action IN [BLOCK, WARN] THEN
    ASYNC revision ← GenerateCompliantRevision(maskedPrompt,
                                              activePolicy,
                                              userTone) // 15.6
    SEND_REVISION_TO_UI(revision)
END IF

// — Phase 5: Logging (always executed) —————
CreateAnonymizedLog( // 15.7
    userId      := userId,
    departmentId := departmentId,
    platform     := platform,
    riskLevel    := decision.riskLevel,
    action       := decision.action,
    categories   := decision.categories,
    classificationSource := finalResult.source
)

END FUNCTION

```

16. Expected Deployment

Global Overview

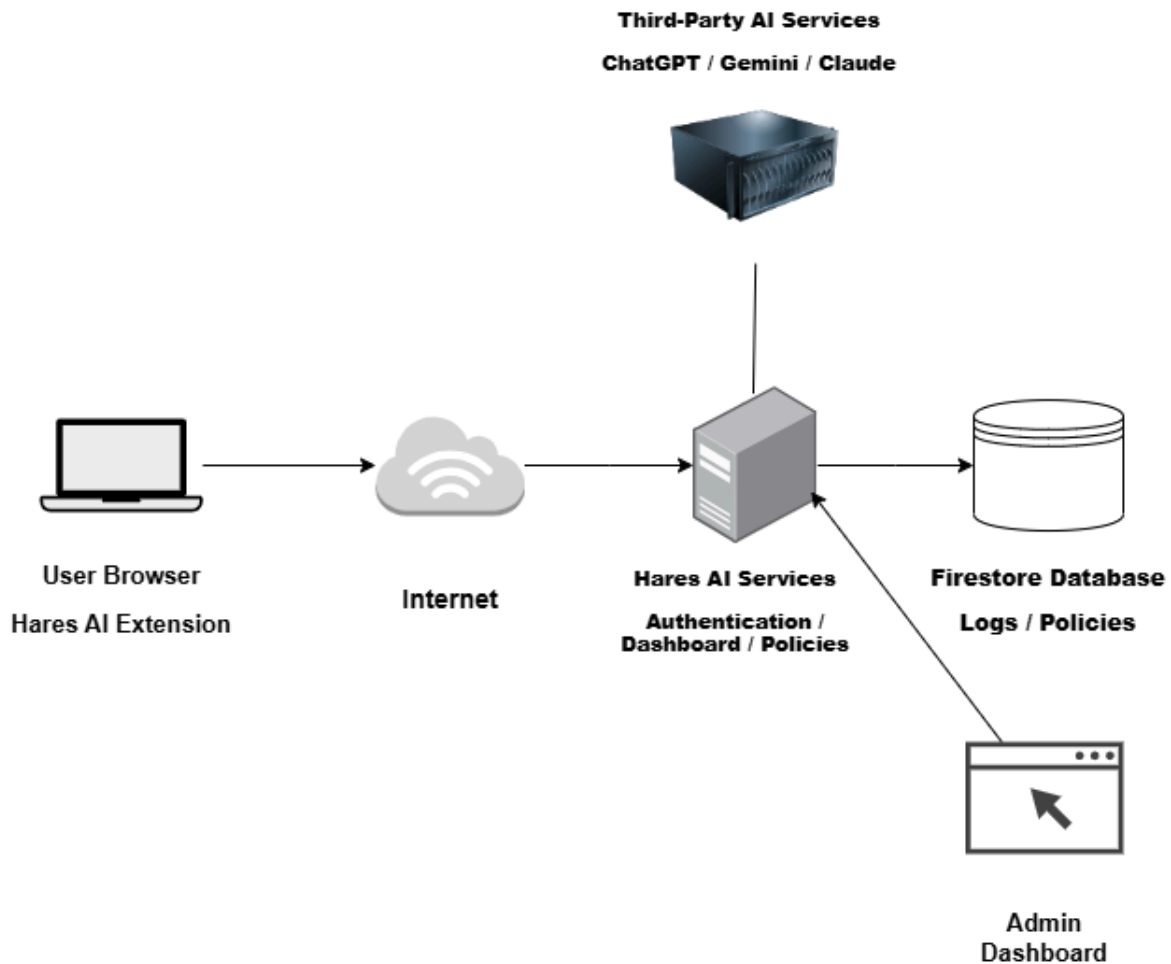


Figure 37. System Deployment Diagram.

The expected deployment of Hares AI consists of six main components: User Browser, Internet, Hares AI Services, Firestore Database, Third-Party AI Services, and Admin Dashboard.

- The User Browser represents the browser used by the employee to access AI platforms. The Hares AI extension is installed in the browser to monitor prompts before they are submitted. The Internet represents the connection between the user browser, Hares AI services, and external services.
- Hares AI Services represent the main system services responsible for authentication, dashboard access, and policy synchronization. Firestore Database is used to store anonymized activity logs, user information, and organization policies.
- Third-Party AI Services represent external AI platforms such as ChatGPT, Gemini, and Claude. The Admin Dashboard represents the web interface used by administrators to view logs, monitor activity, and manage policies.

- The user interacts with AI platforms through the browser where the Hares AI extension is installed. The extension helps detect sensitive data before the prompt is submitted. Hares AI Services communicate with Firestore Database to store anonymized logs and retrieve policies. The Admin Dashboard connects to Hares AI Services so administrators can monitor usage and manage system policies.

17. Test Scenario

We Wanted to test specific functionalities within our system, specifically those related to prompt interception, sensitive data detection, risk classification and activity logging.

The table provided presents the test scenarios required for the system.

These tests are established before the development of the system begins, so the results can not be determined yet.

Test ID	Test Scenario	Test Case	Pre-Condition	Test Steps	Test Data	Expected Result	Post-condition
TC-1	Prompt Interception.	Verify system blocks a prompt containing a Saudi National ID (High Risk).	User is logged in; Hares AI extension is active on ChatGPT; detection rules are enabled.	1. Open ChatGPT in Chrome 2. Type prompt containing a Saudi National ID 3. Attempt to click Send.	Prompt: 'Please verify this customer: Khalid Al-Anazi, ID: 1085736492, account ref #441'.	Extension intercepts the prompt before submission High Risk Blocking overlay appears Detected entity displayed: Saudi National ID: 1085736492 Submission is blocked prompt does NOT reach ChatGPT Block event logged to ActivityLog DB.	Prompt not submitted; block record in ActivityLog.
TC-2	Prompt Interception.	Verify system warns user for a Medium Risk prompt and allows override.	User is logged in; extension active on Claude.ai.	1. Open Claude.ai in Chrome 2. Type prompt referencing internal financial data 3. Attempt to submit 4. On overlay: click 'Submit Anyway'.	Prompt: 'Summarize the Q3 revenue figures from the attached spreadsheet for the leadership memo'.	Medium Risk overlay shown Flagged entity displayed: Financial figures Q3 revenue User has three options: Edit, Submit Anyway, Revise On 'Submit Anyway': prompt is submitted to Claude.ai Event logged with action = WARNED +	Prompt submitted to AI; WARNED log entry with override recorded.

						SUBMITTED.	
TC-3	Prompt Interception.	Verify system shows non-blocking info banner for Low Risk prompts.	User is logged in; extension active; 'Show Low Risk banners' enabled in user settings.	1. Open ChatGPT 2. Type prompt referencing an internal API endpoint 3. Click Send	Prompt: 'Generate test cases for our internal pricing endpoint /api/v2/pricing'.	Prompt is submitted to ChatGPT without blocking A small toast banner appears bottom-right: 'LOW RISK Info' Flagged term shown: Internal endpoint /api/v2/pricing Toast auto-dismisses after 6 seconds Event logged with action = ALLOWED, risk = LOW.	Prompt submitted; LOW risk log entry created.
TC-4	Prompt Interception.	Verify system allows a clean prompt with no sensitive data.	User is logged in; extension active.	1. Open ChatGPT 2. Type a generic, non-sensitive prompt 3. Click Send	Prompt: 'Explain the difference between supervised and unsupervised machine learning'.	No overlay or banner shown Prompt is submitted to ChatGPT immediately Event logged with action = ALLOWED, risk = NONE.	Prompt submitted; NONE risk log entry created.
TC-5	Log Review.	Verify users can view and filter their personal activity log.	User is logged in; at least 1 prompt scan events exist in ActivityLog for this user.	1. Navigate to 'My Activity Log' 2. Set Risk Level filter to 'High' 3. Set Platform filter to 'ChatGPT' 4. Enter keyword 'IBAN' in search bar 5. Click on a row to expand details	Risk Filter: High Platform: ChatGPT Keyword: IBAN.	Table displays only High-risk ChatGPT entries containing 'IBAN' Expanding a row shows full prompt text with flagged entities highlighted All detection details visible in expanded view No data is modified.	No data changed; display only.
TC-6	Log Review.	Verify admin can export the department	Department Admin is logged in; department	1. Navigate to 'Department Log Review'	Date range: Last 7 days No additional filters applied.	System generates a CSV file	CSV file downloaded; no data in the

		activity log as a CSV file.	log has at least 1 entry.	2. Apply date range filter: Last 7 days 3. Click 'Export CSV'.		File is downloaded to the admin's device CSV contains columns: Timestamp, User, Platform, Prompt Preview, Risk Level, Action Taken Only entries matching the active date filter are included.	system is modified.
TC-7	Prompt Revision.	Verify users can generate and use a safe revised version of a blocked prompt.	User is logged in; at least one blocked prompt exists in the user's history.	1. Navigate to 'Blocked Messages' 2. Click 'Revise' on a blocked entry 3. Review the generated revised prompt 4. Click 'Use This Version' Or just observe it.	Original Prompt: 'Draft a refund email to Mariam Al-Hashimi (ID 1102936453, IBAN SA03 8000 0000 6080 1016 7519) for invoice 4421'.	Revision screen shows original prompt with entities highlighted in red Revised prompt shows sensitive data replaced with placeholders: [Customer Name], [removed] Structured alternative version available User clicks 'Use This Version' revised prompt is placed in the AI platform input Revised Prompt record saved in DB.	Revised Prompt record created in DB; prompt ready for submission.
TC-8	Org Dashboard.	Verify System Admin can view organization wide risk statistics.	System Admin is logged in; multiple departments have activity log entries.	1. Navigate to 'Org Dashboard' 2. Review stat cards 3. Review risk trend chart 4. Review stacked bar chart per	N/A.	Stat cards display: Total Users, Total Prompts, High Risk Incidents, Departments Count Risk trend chart shows org-wide	No data modified; dashboard is display-only.

				department		incidents over time Stacked bar chart shows risk distribution per department All values are accurate and up to date.	
TC-9	Role Assignment.	Verify System Admin can assign the Department Admin role to a user.	System Admin is logged in; target user currently has role = USER.	1. Navigate to 'Role Assignment' 2. Search for target user by name or email 3. Change role dropdown from 'User' to 'Department Admin' 4. Confirm the change in the confirmation dialog	Target User: Faisal Al-Zahrani Email: faisal@acmecorp.com New Role: Department Admin.	Confirmation dialog appears before upgrading After confirmation, role updated in User DB User's next login redirects to Department Admin dashboard Role change is logged in the audit trail.	User.role = DEPT_ADMIN in User DB.
TC-10	User Management.	Verify Department Admin can deactivate a user in their department.	Department Admin is logged in; the target user is Active in the same department.	1. Navigate to 'User Management' 2. Locate the target user in the table 3. Click 'Deactivate' 4. Confirm deactivation in the confirmation dialog	Target User: Nora Al-Ghamdi Email: nora@hairesai.com Department: Engineering.	Confirmation dialog: 'Are you sure you want to deactivate Nora Al-Ghamdi?' After confirmation, user status set to Inactive in User DB Status badge changes to red 'Inactive' Deactivated users cannot log in.	User.status = Inactive in User DB.
TC-11	System Configuration.	Verify System Admin can add a new regex detection rule.	System Admin is logged in; System Configuration page is accessible.	1. Navigate to 'System Configuration' 2. Go to the Detection Rules section 3. Click 'Add Rule' 4. Enter rule name, regex	Rule Name: Employee ID Pattern Regex: \bEMP-\d{6}\b Risk Level: Medium.	New rule appears in the Detection Rules table Rule status is Active Extension uses the new pattern on the next prompt	New detection rule persisted in system configuration

				pattern, and risk level 5. Click Save		scan Prompts containing 'EMP-XXXX XX' format are flagged as Medium Risk	
--	--	--	--	--	--	---	--

Table 3. Test Cases.

18. Project Status

Hares AI has successfully completed both the Analysis and Design phases, establishing a solid foundation for the next stage of development. Throughout these phases, the team gained a deeper understanding of browser extension architecture under Manifest V3, sensitive data detection techniques using Regex patterns, and the integration of external AI classification services. The work completed includes the system requirements, use case diagrams and descriptions, analysis and design class diagrams, system architecture, UI mockups, database schema, and core detection algorithms.

In the upcoming semester, the team will shift focus to the Implementation and Testing phases. Key challenges are anticipated around real-time prompt interception across different AI platform DOM structures, achieving the required detection accuracy threshold of 90%, and ensuring the full pipeline remains within the defined latency constraints. The team has already begun preparing for these challenges through research and technical planning.

Phase	Completion	Risks/Issues	Solution
Analysis	100%	Defining and classifying 24 sensitive data categories required extensive research and team alignment	Conducted structured team sessions and finalized categories with advisor feedback
Design	100%	Designing a privacy-first	Adopted a Layered

		architecture within Manifest V3 constraints while maintaining real-time performance	Architecture with an Event-Driven pattern to isolate the detection pipeline from external services
Implementation	0%	DOM structure changes across ChatGPT, Gemini, and Claude may break prompt interception	Flexible platform-specific content scripts with fallback mechanisms will be implemented
Testing	0%	Achieving 90% detection accuracy while keeping false positives below 10% across all categories	A dedicated test dataset will be prepared and detection patterns iteratively refined

Table 4. Project Status.

19. Conclusion

Hares AI addresses an important and growing security challenge caused by the rapid use of AI tools in workplaces. As employees increasingly rely on platforms such as ChatGPT, Gemini, and Claude, organizations face a higher risk of accidental exposure of sensitive information, including passwords, API keys, personal data, financial records, and confidential business content. This project proposes a practical solution that focuses on prevention at the point where the risk begins: before the prompt is submitted to an AI platform.

The system was designed as a lightweight browser extension supported by an admin dashboard and secure backend services. Its main strength is the privacy-first detection approach, where prompt analysis is performed locally using Regex patterns whenever possible. Sensitive data is masked before any further processing, and only anonymized metadata is stored for monitoring and auditing purposes. For ambiguous cases, the system uses the Gemini API only after detected sensitive content has been replaced with placeholder tags, which supports the project's goal of reducing exposure while maintaining useful detection capabilities.

Throughout the project, we defined the system requirements, use cases, analysis classes, interaction diagrams, design classes, architecture, database schema, algorithms, expected deployment, and test scenarios. The proposed architecture separates the presentation layer, business logic layer, data access layer, and data layer, making the system easier to maintain, extend, and test. The event-driven behavior of the browser extension also supports real-time prompt interception while keeping the user experience responsive.

In conclusion, Hares AI provides a clear and feasible solution for protecting organizations from unintentional AI-related data leakage. By combining local detection, risk classification, user alerts, prompt blocking, compliant prompt revision, anonymized logging, and administrative

monitoring, the system balances security, privacy, and usability. Future work may include improving detection accuracy through larger test datasets, expanding support to additional browsers and AI platforms, enhancing policy customization for different departments, and conducting real-world user testing to evaluate performance and usability in organizational environments.

20. Reference

21.1 Glossary

The following glossary defines the key terms and concepts used throughout this document to provide a clear and consistent understanding of their meanings:

Term	Definition
AI Platform	A web-based tool that uses artificial intelligence to generate text responses, such as ChatGPT, Google Gemini, and Claude.ai.
Sensitive Data	Any information that, if exposed, leaked, or misused, may cause harm to an individual or organization (e.g., identity theft, financial fraud, account compromise, privacy violations, legal risks, or security breaches).
Prompt	The text input typed by a user into an AI platform before it is submitted for processing.
Prompt Interception	The process by which the Hares AI extension captures a user's prompt text before it is transmitted to the AI platform, enabling analysis and risk classification.
Risk Level	A classification assigned to a detected prompt: Low, Medium, or High, indicating the severity of potential data exposure.

Regex (Regular Expression)	A sequence of characters that defines a search pattern, used by Hares AI for local detection of sensitive data categories such as national IDs, API keys, and passwords.
Masking	The process of replacing identified sensitive data within a prompt with a placeholder tag (e.g., [API_KEY], [PASSWORD]) before any external transmission.
False Positive	An instance where the system incorrectly flags a non-sensitive piece of data as sensitive, generating an unnecessary alert.
False Negative	An instance where the system fails to detect actual sensitive data, allowing it to pass through without an alert.
Admin Dashboard	A web-based interface accessible to System Admins and Department Admins for reviewing anonymized activity logs, monitoring risk trends, managing users, and configuring detection policies.
Manifest V3	The current security and policy standard for Google Chrome browser extensions, which governs how extensions interact with web pages and browser APIs.
Browser Extension	A software add-on installed in a web browser that augments its functionality, in this case by monitoring and intercepting AI platform prompts.
Content Script	A JavaScript file that runs in the context of a specific web page (e.g., ChatGPT, Gemini), allowing the Hares AI extension to access and modify page elements.
Background Service Worker	An isolated, event-driven script in a Manifest V3 extension that handles tasks like policy enforcement, Gemini API calls, and Firebase communication without direct access to the page DOM.
PII (Personally Identifiable Information)	Any data that can be used to identify a specific individual, such as a national ID, passport number, phone number, or email address.
DLP (Data Loss Prevention)	A category of security tools and practices designed to detect and prevent the unauthorized transmission of sensitive information outside an organization.
Department Admin	A user role with elevated access to view and manage activity logs and users within a specific organizational department.

System Admin	A user role with the highest level of access, responsible for organization-wide configuration, role assignment, and system management.
--------------	--

Table 5. Glossary of Terms.

21.2 Bibliography

The following references include all tools, platforms, academic sources, and standards consulted or cited throughout the development of this project:

- [1] OpenAI, "ChatGPT," OpenAI, [Online]. Available: <https://chat.openai.com>. [Accessed 15 January 2026].
- [2] Google DeepMind, "Google Gemini," Google LLC, [Online]. Available: <https://gemini.google.com>. [Accessed 20 January 2026].
- [3] Anthropic, "Claude," Anthropic, [Online]. Available: <https://claude.ai>. [Accessed 20 January 2026].
- [4] Google LLC, "Chrome Extensions — Manifest V3 Overview," Google Developers, [Online]. Available: <https://developer.chrome.com/docs/extensions/mv3/intro/>. [Accessed 10 February 2026].
- [5] Google LLC, "Chrome Extensions — Message Passing," Google Developers, [Online]. Available: <https://developer.chrome.com/docs/extensions/mv3/messaging/>. [Accessed 12 February 2026].
- [6] Google LLC, "Chrome Storage API," Google Developers, [Online]. Available: <https://developer.chrome.com/docs/extensions/reference/storage/>. [Accessed 12 February 2026].
- [7] Google LLC, "Gemini API Documentation," Google AI for Developers, [Online]. Available: <https://ai.google.dev/docs>. [Accessed 18 February 2026].
- [8] Google LLC, "Firebase Authentication," Firebase, [Online]. Available: <https://firebase.google.com/docs/auth>. [Accessed 5 March 2026].
- [9] Google LLC, "Cloud Firestore Documentation," Firebase, [Online]. Available: <https://firebase.google.com/docs/firestore>. [Accessed 5 March 2026].
- [10] ProtectAI, "LLM Guard," Protect AI, Inc., [Online]. Available: <https://llm-guard.com>. [Accessed 1 March 2026].
- [11] Nightfall AI, "Nightfall AI — AI-Native Data Loss Prevention," Nightfall AI, Inc., [Online]. Available: <https://www.nightfall.ai>. [Accessed 3 March 2026].
- [12] LeakEye, "LeakEye PII Shield," LeakEye, [Online]. Available: <https://chrome.google.com/webstore>. [Accessed 3 March 2026].
- [13] Lakera, "Lakera Guard — Real-Time AI Security," Lakera AI, [Online]. Available: <https://www.lakera.ai>. [Accessed 4 March 2026].
- [14] Visual Paradigm International Ltd., "Visual Paradigm — Agile Modeling and Diagramming Tool," [Online]. Available: <https://www.visual-paradigm.com/>. [Accessed 20 February 2026].
- [15] Figma, Inc., "Figma — The Collaborative Interface Design Tool," [Online]. Available: <https://www.figma.com>. [Accessed 1 March 2026].
- [16] Lucid Software, "Lucidchart — Diagramming Powered by Intelligence," Lucid Software Inc., [Online]. Available: <https://www.lucidchart.com/pages>. [Accessed 5 March 2026].

- [17] Microsoft Corporation, "Microsoft Excel," Microsoft Corporation, Redmond, 2026.
- [18] Zaremba, W. et al., "OpenAI Usage Policies," OpenAI, [Online]. Available: <https://openai.com/policies/usage-policies>. [Accessed 10 February 2026].
- [19] National Institute of Standards and Technology (NIST), "NIST Special Publication 800-122: Guide to Protecting the Confidentiality of Personally Identifiable Information (PII)," U.S. Department of Commerce, Gaithersburg, MD, 2010. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-122/final>.
- [20] Mozilla Developer Network, "Regular Expressions," MDN Web Docs, Mozilla, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions. [Accessed 15 February 2026].